

LAGOS STATE UNIVERSITY

FACULTY OF COMPUTING

DEPARTMENT OF COMPUTER SCIENCE

DESIGN AND IMPLEMENTATION OF A WEB-BASED FLEET MANAGEMENT SYSTEM FOR LEKKI FREE ZONE DEVELOPMENT COMPANY

A SOFTWARE PROJECT REPORT

Submitted to the Department of Computer Science, Faculty of Computing, Lagos State University, in partial fulfilment of the requirements for the award of a degree in Computer Science.

BY

AYASI BOLUATIFE ANTHONY

MATRIC NUMBER: 2305-9-1-0-4-5

SUPERVISOR: _____

SUBMISSION DATE: _____

DECLARATION

I, Ayasi Boluatife Anthony, declare that this report describes the software system submitted for assessment and that the project materials have been organized to permit review, local execution and database inspection. Sources consulted in the design and documentation are acknowledged in the references section.

Student Signature: _____

Date: _____

CERTIFICATION

This is to certify that the project titled “Design and Implementation of a Web-Based Fleet Management System for Lekki Free Zone Development Company” was carried out by Ayasi Boluatife Anthony, matric number 2305-9-1-0-4-5, in the Department of Computer Science, Faculty of Computing, Lagos State University.

Supervisor: _____

Signature: _____

Date: _____

ACKNOWLEDGEMENTS

I acknowledge the academic guidance, feedback and learning opportunities that supported the completion of this project. I also appreciate the staff and operational context of Lekki Free Zone Development Company that informed the practical requirements represented in the system. Finally, I am grateful to my family, colleagues and lecturers for their encouragement throughout the design, implementation and documentation process.

ABSTRACT

Fleet operations generate interconnected records for vehicles, drivers, assignments, trips, fuel consumption, maintenance, incidents and compliance documents. When these records are fragmented across spreadsheets or paper files, it becomes difficult to determine fleet readiness, trace operational history, control access or produce reliable management reports. This project presents the design and implementation of a web-based Fleet Management System for Lekki Free Zone Development Company. The application was developed with React and TypeScript, built with Vite, and connected to a Supabase backend comprising PostgreSQL, Authentication, Storage and Edge Functions. It provides role-based access for system administrators, editors and end users; supports vehicle and driver registers; records trips, fuel, maintenance, incidents and odometer changes; manages documents; generates filtered reports; and imports historical Excel workbooks through a preview, mapping and validation workflow. Security controls include authenticated sessions, optional time-based one-time-password multi-factor authentication, database Row Level Security and server-side authorization for privileged user operations. The submitted database package contains numbered SQL scripts and fictional sample records for academic inspection. Evaluation focused on functional behavior, validation, permissions, duplicate handling, build quality and usability. The resulting system centralizes fleet information, preserves operational history and offers a clearer basis for day-to-day decisions and future extension.

Keywords: fleet management, React, Supabase, PostgreSQL, role-based access control, Excel import, fleet reporting.

TABLE OF CONTENTS

CHAPTER ONE: INTRODUCTION....1
1.1 Background of the Study....1
1.2 Statement of the Problem....2
1.3 Aim and Objectives....2
1.4 Scope of the Project....2
1.5 Significance of the Project....2
1.6 Methodology....3
1.7 Organization of the Report....3
CHAPTER TWO: REVIEW OF RELEVANT CONCEPTS AND TECHNOLOGIES....4
2.1 Fleet Information Management....4
2.2 Limitations of Spreadsheet-Only Processes....4
2.3 Component-Based Web Development....5
2.4 Relational Data and Integrity....5
2.5 Backend-as-a-Service Architecture....5
2.6 Authentication, MFA and Role-Based Access....5
2.7 Historical Data Import....5
2.8 Summary of the Review....6
CHAPTER THREE: SYSTEM ANALYSIS AND DESIGN....7
3.1 Analysis of the Existing Process....7
3.2 Proposed System....7
3.3 Functional Requirements....7
3.4 Non-Functional Requirements....8
3.5 System Architecture....8
3.6 User Roles and Access Design....9
3.7 Database Design....9
3.8 Import Workflow Design....10
3.9 Security Design....11
3.10 Interface Design Principles....11

CONTENTS (CONTINUED)

CHAPTER FOUR: IMPLEMENTATION **12**
4.1 Development Environment and Technology Stack 12
4.2 Project Structure 12
4.3 Authentication and Session Handling 12
4.4 Fleet Overview Dashboard 13
4.5 Operational Registers 14
4.6 Assignment and Odometer Rules 15
4.7 Maintenance and Incident Handling 15
4.8 Data Import Module 15
4.9 Reporting and Export 16
4.10 Document and Notification Modules 17
4.11 User Administration 18
4.12 Deployment and Portable Submission 18
4.13 Selected Interface Source Code 18
CHAPTER FIVE: TESTING AND EVALUATION **22**
5.1 Testing Strategy 22
5.2 Test Cases and Outcomes 22
5.3 Build and Source Verification 23
5.4 Security Evaluation 23
5.5 Usability Evaluation 23
5.6 Limitations 23
CHAPTER SIX: USER GUIDE AND INSTALLATION **24**
6.1 Requirements 24
6.2 Running the Submitted Code 24
6.3 Signing In 24
6.4 Navigating the System 25
6.5 Dashboard 25
6.6 Managing Vehicles and Drivers 25
6.7 Recording Operational Activity 31
6.8 Importing Existing Records 37
6.9 Reports and Exports 37
6.10 Notifications and Settings 37
6.11 User Administration 37
6.12 Troubleshooting 38
6.13 Safe Submission and Demonstration 38
CHAPTER SEVEN: CONCLUSION **39**
7.1 Conclusion 39
7.2 Recommendations 39
7.3 Contribution of the Project 40
REFERENCES **41**
APPENDIX A: DATABASE RESTORATION PACKAGE **42**
APPENDIX B: CUSTOM API ENDPOINTS **43**
APPENDIX C: SUBMISSION CHECKLIST **44**

LIST OF FIGURES

Figure 3.1 High-level system architecture

Figure 3.2 Simplified data relationship model

Figure 3.3 Historical workbook import workflow

Figure 4.1 Sign-in interface

Figure 4.2 Fleet overview dashboard

Figure 4.3 Vehicle register

Figure 4.4 Three-step data import interface

Figure 4.5 Reports and analytics interface

Figure 4.6 User access administration interface

Figure 4.7 Dashboard interface composition code

Figure 4.8 Vehicle validation and save code

Figure 4.9 Three-stage import interface code

Figure 6.1 Add New Vehicle form before data entry

Figure 6.2 Completed vehicle form with additional details

Figure 6.3 Newly added vehicle in the register

Figure 6.4 Completed driver registration form

Figure 6.5 Newly added driver in the register

Figure 6.6 Completed trip form

Figure 6.7 Logged trip in the trip history

Figure 6.8 Completed maintenance scheduling form

Figure 6.9 Scheduled work in the maintenance register

Figure 6.10 Completed incident report form

Figure 6.11 Reported incident in the incident register

Note: interface screenshots use fictional demonstration data and do not expose live account credentials or operational records.

LIST OF TABLES

Table 1.1 Project objectives

Table 2.1 Existing and proposed process comparison

Table 3.1 Functional requirements

Table 3.2 Non-functional requirements

Table 3.3 Role and permission matrix

Table 3.4 Database table summary

Table 4.1 Technology stack

Table 4.2 Application modules

Table 5.1 System test cases and outcomes

Table 6.1 Common operating problems and responses

Table A.1 Database restoration sequence

Table B.1 Custom Edge Function endpoints

ABBREVIATIONS

Abbreviation	Meaning
AAL	Authenticator Assurance Level
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
JWT	JSON Web Token
LFZDC	Lekki Free Zone Development Company
MFA	Multi-Factor Authentication
RBAC	Role-Based Access Control
RLS	Row Level Security
SQL	Structured Query Language
TOTP	Time-Based One-Time Password
UI/UX	User Interface and User Experience
VIN	Vehicle Identification Number

CHAPTER ONE

INTRODUCTION

This chapter establishes the operational problem, project purpose, objectives, scope and significance.

1.1 Background of the Study

A fleet is not only a collection of vehicles. It is an operational system made up of assets, licensed personnel, journeys, fuel expenditure, maintenance work, incidents, supporting documents and management decisions. The value of fleet information depends on whether these records can be connected. A fuel entry without a vehicle reference, a trip without an assigned driver, or a maintenance record that does not update readiness creates an incomplete operational picture.

In many organizations, historical records emerge through separate spreadsheets, paper forms and informal communication. Such methods may work at a small scale, but they make it difficult to answer routine questions consistently: Which vehicles are active? Which drivers are currently assigned? What is the cost of a vehicle over a selected period? Which service tasks are overdue? Which documents or licences are approaching expiry? A central information system provides a shared source of truth and a traceable record of changes.

The LFZDC Fleet Management System was therefore designed as an internal web application that brings operational fleet records into one structured environment. It combines a focused interface with a relational database and controlled user access. The design also recognizes that an organization may already possess years of spreadsheet records before the application is introduced; consequently, historical data import is treated as a core workflow rather than an afterthought.

1.2 Statement of the Problem

The primary problem addressed by this project is fragmented fleet information. When vehicle, driver, fuel, maintenance, trip and incident records are stored in unrelated files, there is no reliable mechanism for enforcing identifiers, preventing duplication, preserving assignments or producing fleet-wide reports. Manual consolidation consumes time, increases the likelihood of transcription errors and makes access control difficult to enforce.

A second problem is continuity. Introducing a new information system should not force operators to abandon useful historical records. The solution must accept existing Excel or exported Google Sheets data, explain validation problems in ordinary language, and avoid writing to the database until the operator has reviewed the proposed import.

1.3 Aim and Objectives

The aim of the project is to design and implement a secure, usable and maintainable web-based system for managing LFZDC fleet assets and operational records.

No.	Objective
1	Create centralized registers for vehicles and drivers.
2	Record assignments, trips, fuel usage, maintenance, incidents and odometer changes.
3	Provide an operational dashboard and filtered management reports.
4	Control access through authentication, role permissions and database policies.
5	Support safe import of historical Excel records with preview, mapping and validation.
6	Maintain document metadata and uploaded files associated with fleet entities.
7	Provide a portable database package and technical documentation for academic assessment.

1.4 Scope of the Project

The implemented scope covers internal fleet record management. It includes dashboard summaries, vehicle and driver records, assignment history, journey logs, fuel entries, maintenance work, incident records, notifications, documents, reports, historical workbook imports, personal settings and administrator user access. The application is responsive and can be opened through a modern desktop or mobile browser.

The project does not include vehicle GPS tracking, live telematics, payroll, procurement approval, public customer access, insurance-claim integration or a native mobile application. Email notification delivery is optional and depends on hosted server configuration. The system records information supplied by authorized operators; it does not independently verify external vehicle or licence databases.

1.5 Significance of the Project

- Operational staff gain a single location for daily fleet records and status checks.
- Managers can compare costs, distance and readiness without manually combining spreadsheets.
- Historical records can be migrated in controlled batches and reviewed before saving.
- Role-based access reduces the risk of unauthorized operational changes.
- The database schema preserves relationships and supports future modules or reporting requirements.
- The submission package can be inspected and restored independently of the hosted interface.

1.6 Methodology

The project followed an iterative development approach. Operational entities and workflows were first identified from the intended fleet context. The database schema and permission model were then designed around stable identifiers and historical relationships. Interface modules were implemented incrementally and tested against fictional operational records. The data importer was refined after considering the common difficulty of matching legacy sheet names and column headings. Finally, the codebase, database scripts and documentation were reorganized for deployment and academic submission.

1.7 Organization of the Report

Chapter One introduces the project. Chapter Two discusses relevant concepts and the technology context. Chapter Three presents requirements and system design. Chapter Four explains implementation and the completed interfaces. Chapter Five records testing and evaluation. Chapter Six serves as the user and installation guide. Chapter Seven concludes the report and proposes future work. Appendices summarize database restoration, API endpoints and submission contents.

CHAPTER TWO

REVIEW OF RELEVANT CONCEPTS AND TECHNOLOGIES

This chapter relates the project to fleet information management, database integrity, web architecture and access control.

2.1 Fleet Information Management

Fleet management systems coordinate asset availability, driver responsibility, journeys, operating expenditure, service activity and compliance records. A useful system must do more than store separate lists. It must show the relationship between an asset and the events that affect its lifecycle. This relational view supports accountability because an operator can move from a vehicle to its assignments, trips, fuel history, incidents, maintenance work and documents.

Operational status is time-sensitive. A vehicle may be active, idle, under maintenance or decommissioned. A driver may be active, on leave, suspended or inactive. The system therefore treats status as controlled business information and closes active assignments when lifecycle changes require it, rather than deleting history.

2.2 Limitations of Spreadsheet-Only Processes

Spreadsheets are flexible and familiar, which makes them suitable for small registers and data exchange. Their flexibility, however, can also produce inconsistent spellings, repeated identifiers, missing references and changes without a shared audit trail. Separate workbooks do not automatically enforce that every trip refers to an existing vehicle and driver. They also do not naturally apply different permissions to administrators, record editors and read-only users.

Area	Spreadsheet-only process	Proposed system
Identity	Plate or licence values may be typed differently across files.	Normalized identifiers connect related records.
Validation	Errors are commonly found after manual review.	Validation occurs before saving and reports the affected row.
Permissions	File access often grants broad editing rights.	Roles and database policies control permitted actions.
History	Overwriting cells can remove previous states.	Assignments and events are kept as dated records.
Reporting	Data must be merged and recalculated manually.	Filters and charts are generated from the shared database.
Migration	Old files remain isolated.	Excel workbooks can be previewed and imported in batches.

2.3 Component-Based Web Development

React structures an interface as reusable components that combine appearance and behavior. This approach supports consistency across navigation, tables, dialogs, forms, status badges and responsive layouts. TypeScript adds explicit data types for vehicle, driver and operational records, reducing ambiguity between interface data and database rows. React Router provides client-side navigation while Vite supplies the development server and optimized production build process [1] [2].

2.4 Relational Data and Integrity

PostgreSQL provides tables, keys, constraints, functions and triggers for structured data. Primary keys uniquely identify records, foreign keys express relationships, and constraints reject invalid values. These mechanisms are particularly important in fleet systems because operational records should not become detached from the vehicles or drivers they describe. PostgreSQL also supports Row Level Security, allowing access rules to be evaluated as part of database queries [3][4].

2.5 Backend-as-a-Service Architecture

Supabase provides a PostgreSQL database together with authentication, object storage, generated data APIs and server-side Edge Functions. For this project, that combination removes the need for a separate Express server while preserving database-level authorization. The browser uses a publishable anonymous key, but actual access depends on the authenticated session and Row Level Security. Privileged operations use server-side functions and never expose the service-role credential in frontend code [5][6].

2.6 Authentication, MFA and Role-Based Access

Authentication establishes who a user is; authorization determines what that user may do. The application separates these concerns. Supabase Auth manages email/password sessions and TOTP multi-factor authentication. The public profile table stores application role and status. The interface hides actions a role cannot use, but security does not rely on hidden buttons: database policies and Edge Function checks enforce the same restrictions. This layered approach follows the principle that client-side controls improve usability but should not be treated as the final security boundary.

2.7 Historical Data Import

A migration tool must protect the quality of the destination system. The implemented importer reads Excel workbooks in the browser, recognizes standard and common legacy sheet names,

suggests field mappings, validates rows, links data through plate and licence identifiers and skips duplicates when appropriate. The operator reviews the findings before import. This separation between parsing and writing makes the process easier to understand and reduces accidental database changes.

2.8 Summary of the Review

The review supports a browser-based relational solution with controlled access and an explicit migration workflow. React and Vite suit a responsive modular interface; PostgreSQL provides integrity and history; Supabase combines the required backend services; and Excel remains an appropriate interchange format for legacy records. These choices directly inform the design presented in Chapter Three.

CHAPTER THREE

SYSTEM ANALYSIS AND DESIGN

This chapter translates the operational need into requirements, roles, architecture, data relationships and safety rules.

3.1 Analysis of the Existing Process

The assumed existing process depends on independent registers maintained at different times. A vehicle may appear under multiple plate-number formats, driver information may be repeated in trip sheets, and cost records may not share the same period or identifier rules. Retrieving a complete vehicle history therefore requires manual searching and reconciliation. The process is also vulnerable to duplicate entry and unclear ownership of changes.

3.2 Proposed System

The proposed system centralizes operational records while preserving the simplicity of browser-based forms and tables. A shared navigation model organizes modules into fleet operations and records/control areas. The dashboard summarizes availability, expenditure and service activity. Detail pages expose history without encouraging destructive deletion. Administrators manage accounts and roles; editors manage operational records; end users can review information without altering it.

3.3 Functional Requirements

ID	Requirement	Priority
FR-01	Authenticate staff with email and password and restore valid sessions.	High
FR-02	Challenge enrolled accounts for TOTP MFA where AAL2 is required.	High
FR-03	Create and view vehicle and driver records with controlled statuses.	High
FR-04	Assign drivers to vehicles while closing conflicting active assignments.	High
FR-05	Record trips, fuel, maintenance, incidents and odometer readings.	High
FR-06	Upload and categorize vehicle or driver documents.	Medium
FR-07	Generate dashboards, filters, charts and downloadable reports.	High
FR-08	Import supported Excel workbooks through preview and validation.	High
FR-09	Store in-app notifications and optionally send email alerts.	Medium

ID	Requirement	Priority
FR-10	Permit system administrators to create, update and remove user accounts safely.	High

3.4 Non-Functional Requirements

Area	Requirement
Usability	Use clear language, consistent controls, responsive layouts and visible feedback.
Security	Protect data through authenticated sessions, RLS, role checks and server-held secrets.
Integrity	Preserve relationships, identifiers and operational history; avoid unsafe deletion.
Performance	Load modules independently and aggregate only the records required for each screen.
Maintainability	Separate UI, data access, business rules, types and backend scripts.
Portability	Provide environment examples, reproducible installation steps and numbered SQL scripts.
Auditability	Retain import batch outcomes and dated operational records.

3.5 System Architecture

The application uses a client-to-service architecture. React renders the interface and invokes typed data modules. The Supabase client manages sessions and communicates with Auth, PostgREST, Storage and Edge Functions. PostgreSQL stores structured records and evaluates RLS policies. The Storage service holds uploaded files, while Edge Functions perform user-administration and notification-email operations that require protected server credentials.

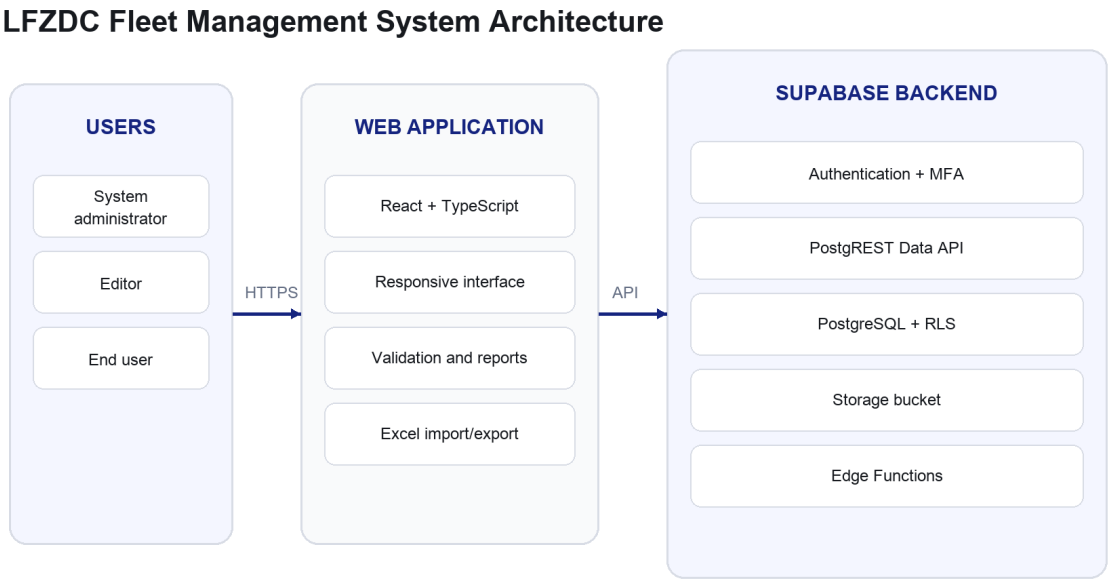


Figure 3.1: High-level system architecture. Source: Author's design, 2026.

3.6 User Roles and Access Design

Capability	System Admin	Editor	End User
View operational records	Yes	Yes	Yes
Create/update fleet records	Yes	Yes	No
Import historical workbooks	Yes	Yes	No
Generate reports	Yes	Yes	Yes
Manage own profile/settings	Yes	Yes	Yes
Create, change or delete users	Yes	No	No

The system prevents an administrator from deleting their own account and prevents deletion of the last remaining system administrator. These rules protect continued administrative access. Vehicle and driver deletion is intentionally unavailable; status changes preserve historical references.

3.7 Database Design

The database is organized around vehicles, drivers and dated operational events. The profiles table extends Supabase Auth with application role and status. Vehicles and drivers are referenced by assignments and activity tables. Nullable foreign keys with appropriate delete behavior preserve records when a relationship is no longer active. Updated-at triggers maintain modification timestamps on supported tables.

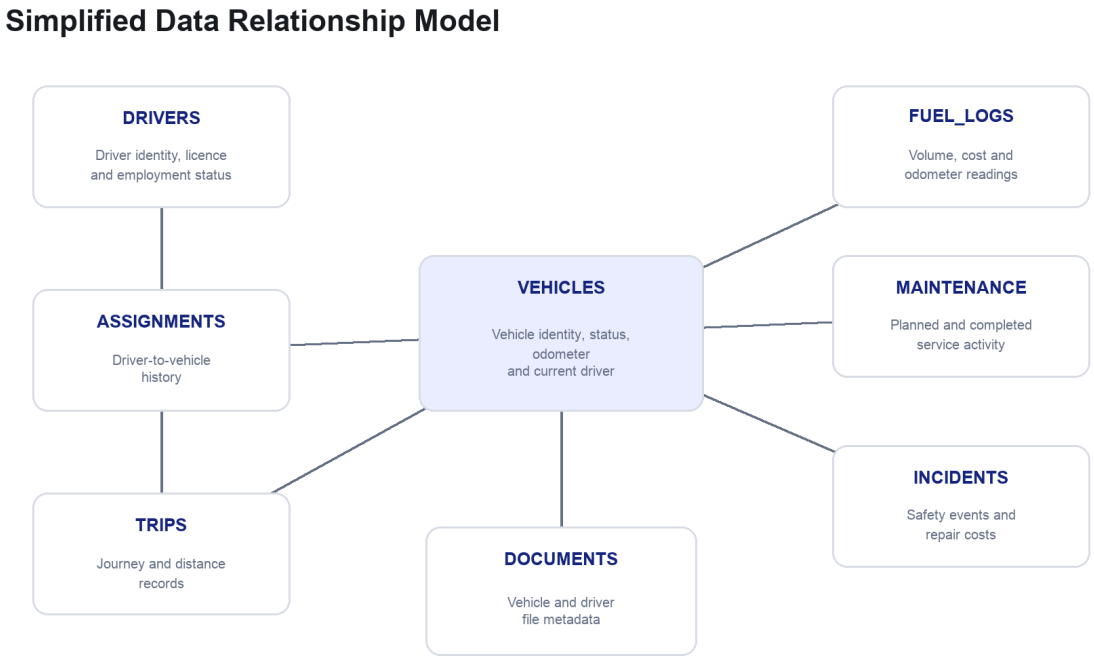


Figure 3.2: Simplified data relationship model. Source: Author's design, 2026.

Table	Purpose
profiles	Application identity, role, department, status and settings.
vehicles	Vehicle master data, lifecycle status, current driver and service mileage.
drivers	Driver identity, contact, licence, employment and status information.
assignments	Dated vehicle-to-driver assignment history.
trips	Journey date, route, purpose and distance.
fuel_logs	Fuel quantity, cost, station and odometer reading.
maintenance	Scheduled/completed service work, provider and cost.
incidents	Incident severity, status, location and repair cost.
documents	Uploaded-file metadata and related vehicle or driver.
notifications	In-app operational alerts and read state.
vehicle_odometer_logs	Mileage history and source record.
import_batches	Workbook filename, counts, sheet summaries and row issues.

3.8 Import Workflow Design

The import process deliberately separates file selection, review and database writing. Operators can download a template, select a workbook, review automatically detected sheets and resolve field mappings. Blocking errors prevent import. Warnings remain visible but do not necessarily stop valid rows. Duplicate rules protect existing records, and the final batch summary records created, skipped and problematic rows.

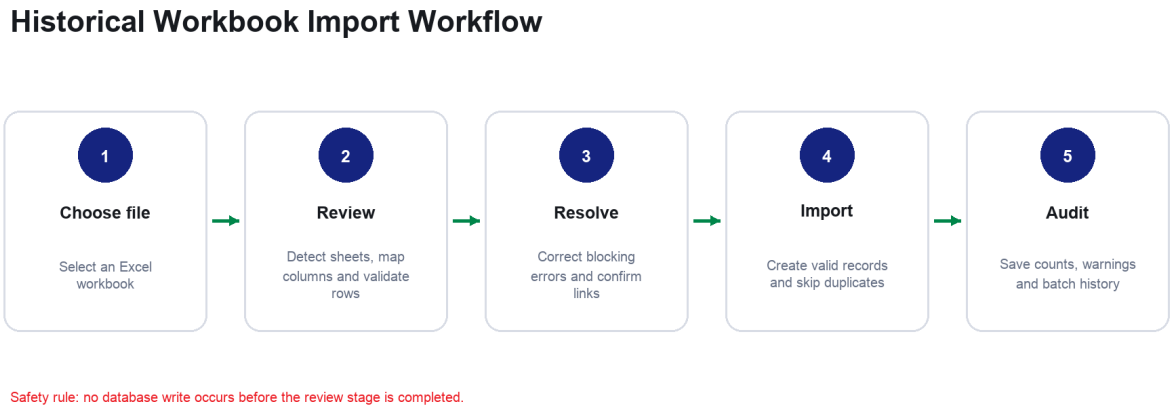


Figure 3.3: Historical workbook import workflow. Source: Author's design, 2026.

3.9 Security Design

- Supabase Auth issues and refreshes browser sessions; passwords are not stored in application tables.
- MFA-aware routes require the correct authenticator assurance level before protected access.
- RLS policies enforce read/write rules at the PostgreSQL layer.
- The frontend uses only the publishable anonymous key; service-role and SMTP credentials remain server-side.
- Privileged Edge Functions validate the caller and profile role before executing administrative actions.
- Submitted environment files contain placeholders and exclude credentials.

3.10 Interface Design Principles

The final interface uses LFZDC navy, red and green selectively rather than as decoration. Navigation collapses to icons to preserve working space and expands when needed. Each page begins with a direct title and action area, followed by a register, form or analytical view. Status colors are used consistently: green for active or completed states, amber for attention, red for urgent conditions and navy for primary actions. The data import interface was simplified to three visible stages so that an average operator can understand what happens next.

CHAPTER FOUR

SYSTEM IMPLEMENTATION

This chapter describes the implemented technology, modules, business rules and interface evidence.

4.1 Development Environment and Technology Stack

Technology	Use in the project
React 19 + TypeScript	Component-based interface and typed application models.
Vite 8	Development server, module transformation and production bundling.
Tailwind CSS 4	Responsive styling and reusable design tokens.
React Router	Protected client-side routes and module navigation.
Supabase JavaScript Client	Authentication, data, Storage and function invocation.
PostgreSQL	Relational records, constraints, functions, triggers and RLS.
Supabase Edge Functions	Privileged user operations and optional email notification delivery.
Recharts	Dashboard and report charts.
SheetJS	Excel workbook parsing, templates, import and export.
Vercel	Hosted frontend deployment.

4.2 Project Structure

The source tree separates application concerns. Shared controls and the layout are located under src/components; authentication state is held in src/context; pages represent complete modules; src/lib contains data access, import, export, rules and utilities; shared types are under src/types; public contains the LFZDC identity assets; supabase contains working SQL and Edge Functions; database-submission contains the numbered restoration package; and docs contains backend and API references.

4.3 Authentication and Session Handling

The sign-in page calls Supabase email/password authentication and waits for the profile record to load. AuthContext observes session changes, restores sessions, normalizes legacy roles and checks MFA assurance. Enrolled users must complete a six-digit authenticator challenge before entering protected routes. Password recovery returns the user to the login route, verifies MFA when required and enforces a minimum password length. Network failures are translated into clearer operational messages.

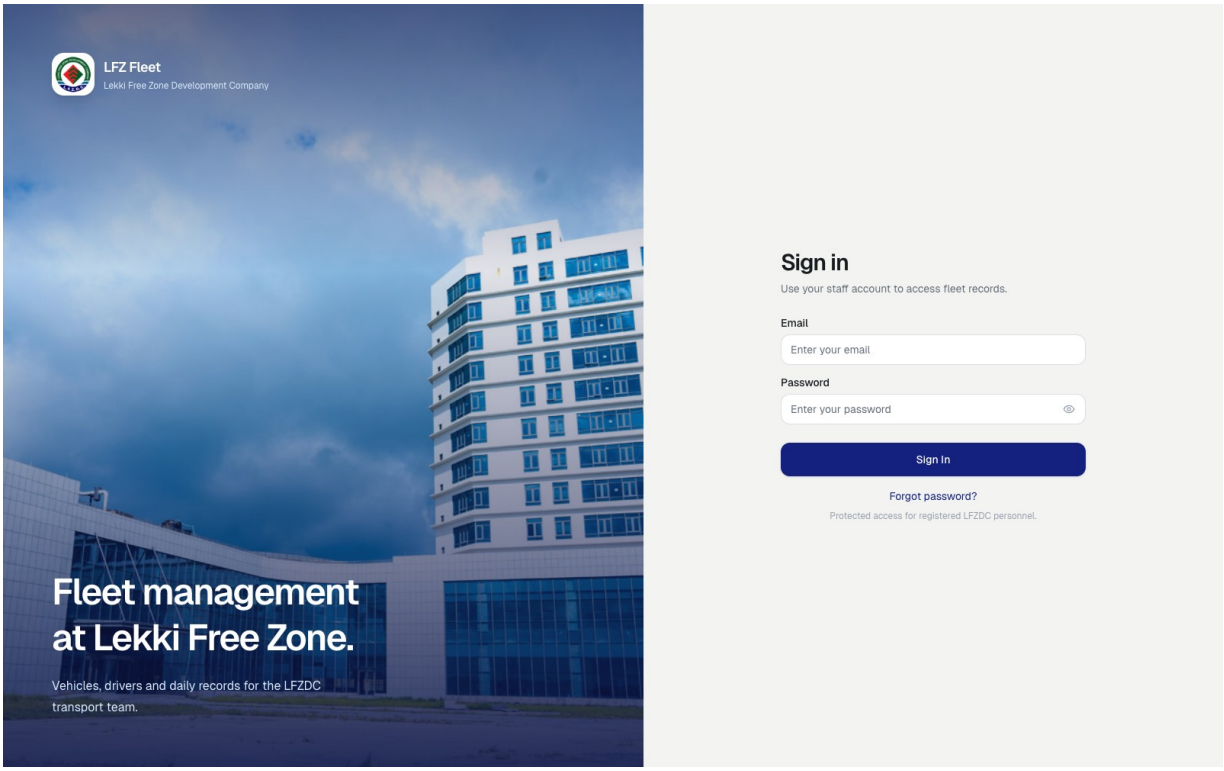


Figure 4.1: Sign-in interface. Source: Author's implementation, 2026.

4.4 Fleet Overview Dashboard

The dashboard aggregates vehicle and driver readiness, current-month fuel and maintenance cost, six-month fuel expenditure, fleet status distribution, recent maintenance and operational notices. Summary cards are links rather than passive decoration, enabling the user to move directly from a metric to the corresponding register. The dashboard handles loading and backend errors explicitly.

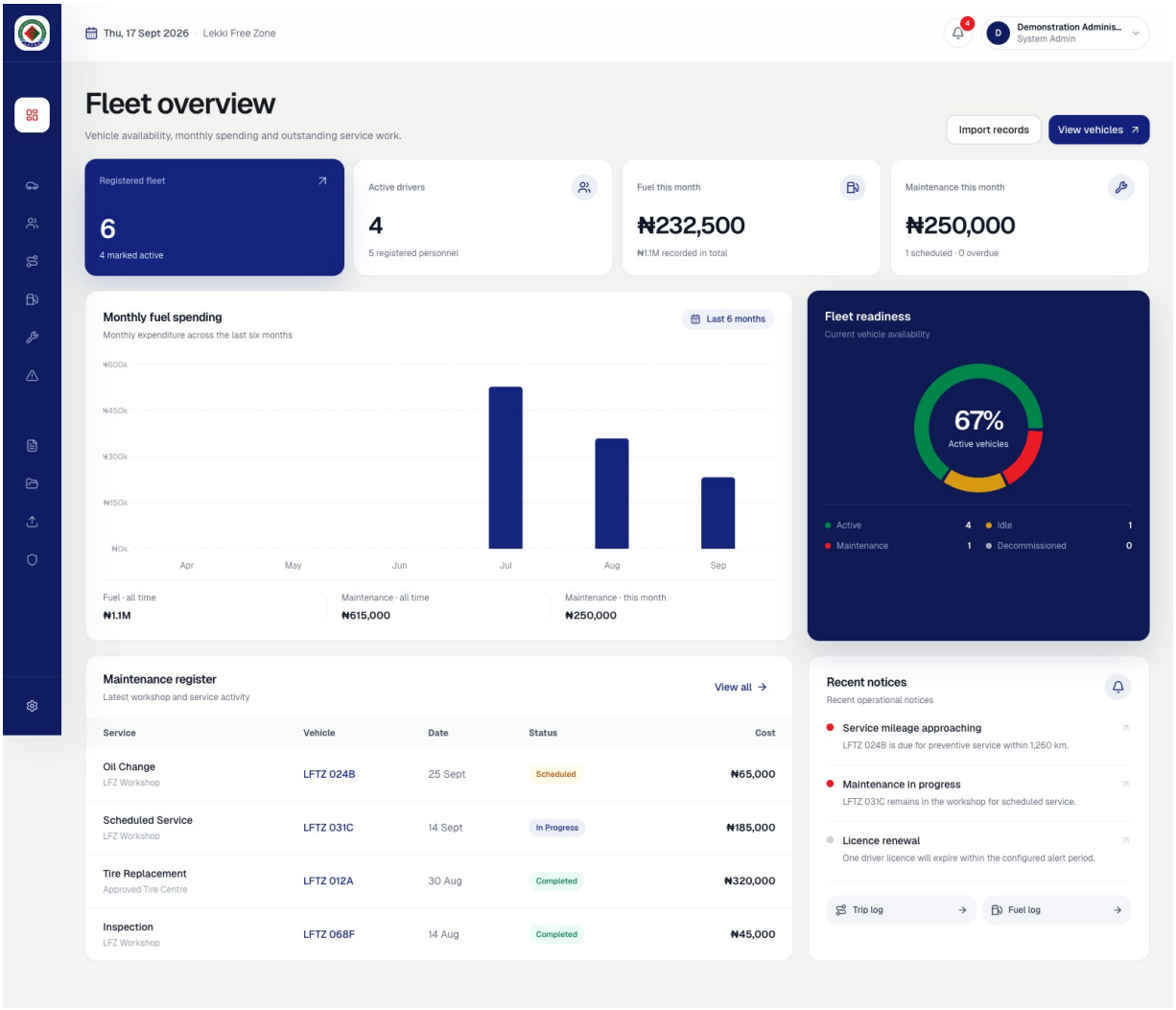


Figure 4.2: Fleet overview dashboard using fictional demonstration records. Source: Author's implementation, 2026.

4.5 Operational Registers

Vehicle and driver registers provide search, filters, pagination and controlled creation. Vehicle status can be changed only to supported lifecycle states. "Under Maintenance" is derived from active maintenance records and is not treated as an arbitrary manual status. Vehicle and driver detail pages combine master data with assignment and activity history. Trips, fuel, maintenance and incidents use comparable table and dialog patterns so operators do not need to learn a different interaction for each record type.

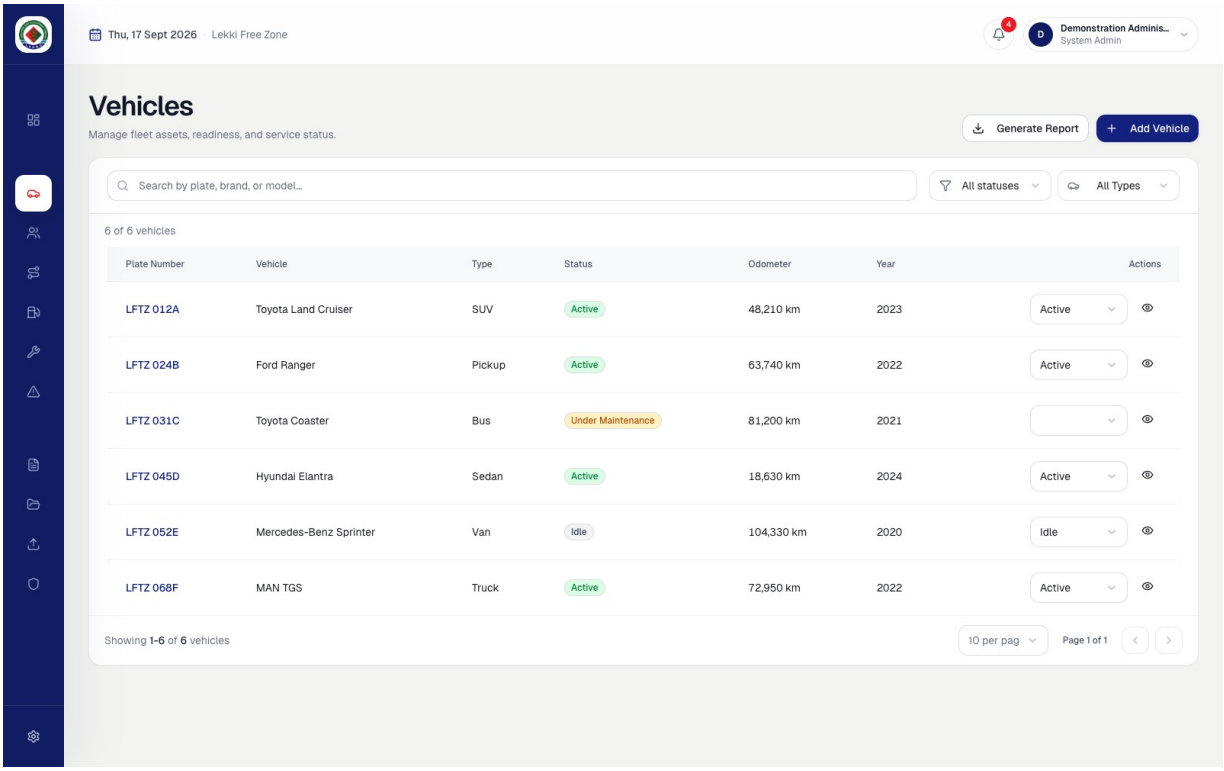


Figure 4.3: Vehicle register using fictional demonstration records. Source: Author's implementation, 2026.

4.6 Assignment and Odometer Rules

Assigning a driver first closes any conflicting active assignment for either the vehicle or driver, creates the new assignment and synchronizes the vehicle's current driver. Mileage can originate from direct readings, fuel entries, trips or maintenance. Each reading is recorded with its source; only a higher reading replaces the vehicle's current odometer. This protects the register from accidental rollback while retaining the submitted reading for review.

4.7 Maintenance and Incident Handling

Maintenance records may be scheduled, in progress, completed or overdue. Active work can place the vehicle in an under-maintenance state, while completion returns it to service only when no other active maintenance remains. Incident records include severity, workflow status, location and preliminary or final repair cost. Incident creation can produce an in-app notification without making external email delivery part of the main transaction.

4.8 Data Import Module

The current import page presents three clear stages: choose file, review and import. Standard .xlsx and .xls files are accepted; users of Google Sheets are instructed to download an Excel copy first. No data is saved during review. The importer detects vehicles, drivers, assignments, fuel logs,

maintenance, incidents and trips, and can suggest mappings for unusual legacy headings. Import history stores the file name, operator, timestamp, counts and row-level issues.

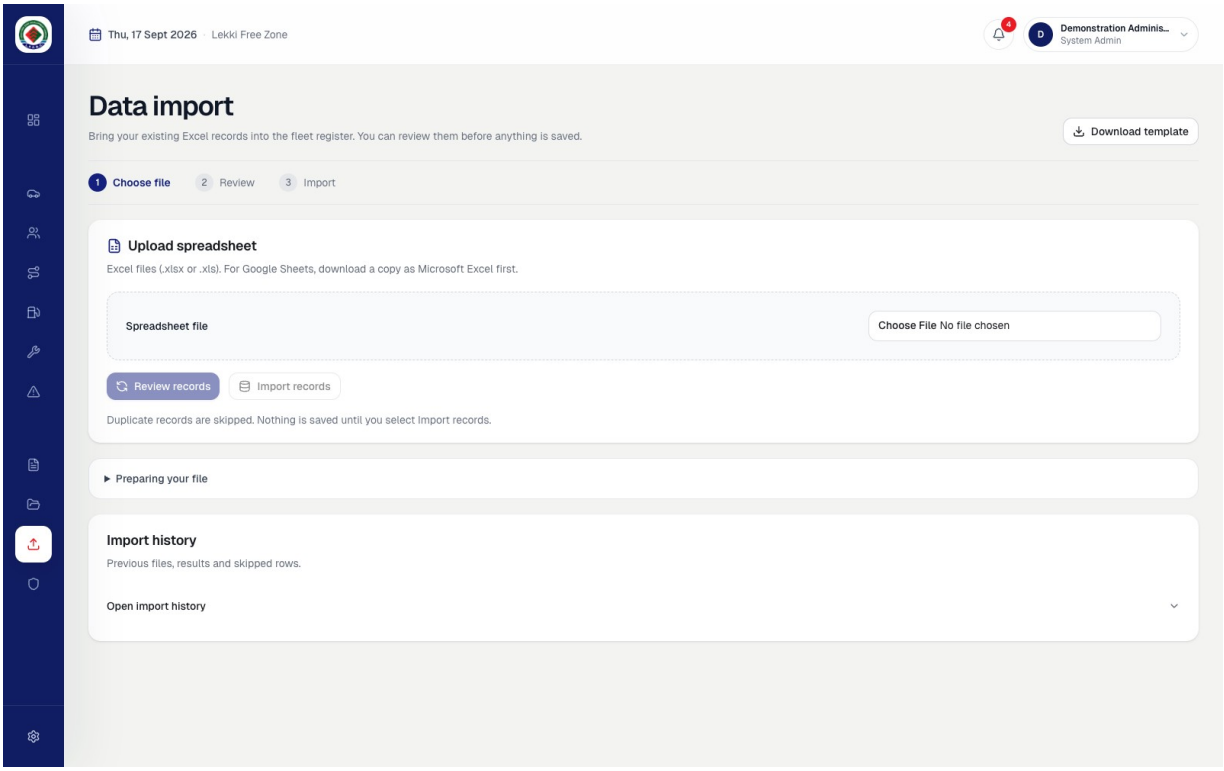


Figure 4.4: Three-step data import interface. Source: Author's implementation, 2026.

4.9 Reporting and Export

The reports module filters by vehicle and period, summarizes fuel, maintenance, incident and distance metrics, and visualizes cost distribution, trends and vehicle comparison. The displayed report can be exported as a PDF. Registers also provide targeted Excel exports for operational follow-up, such as maintenance-overdue or warranty-expiry views.

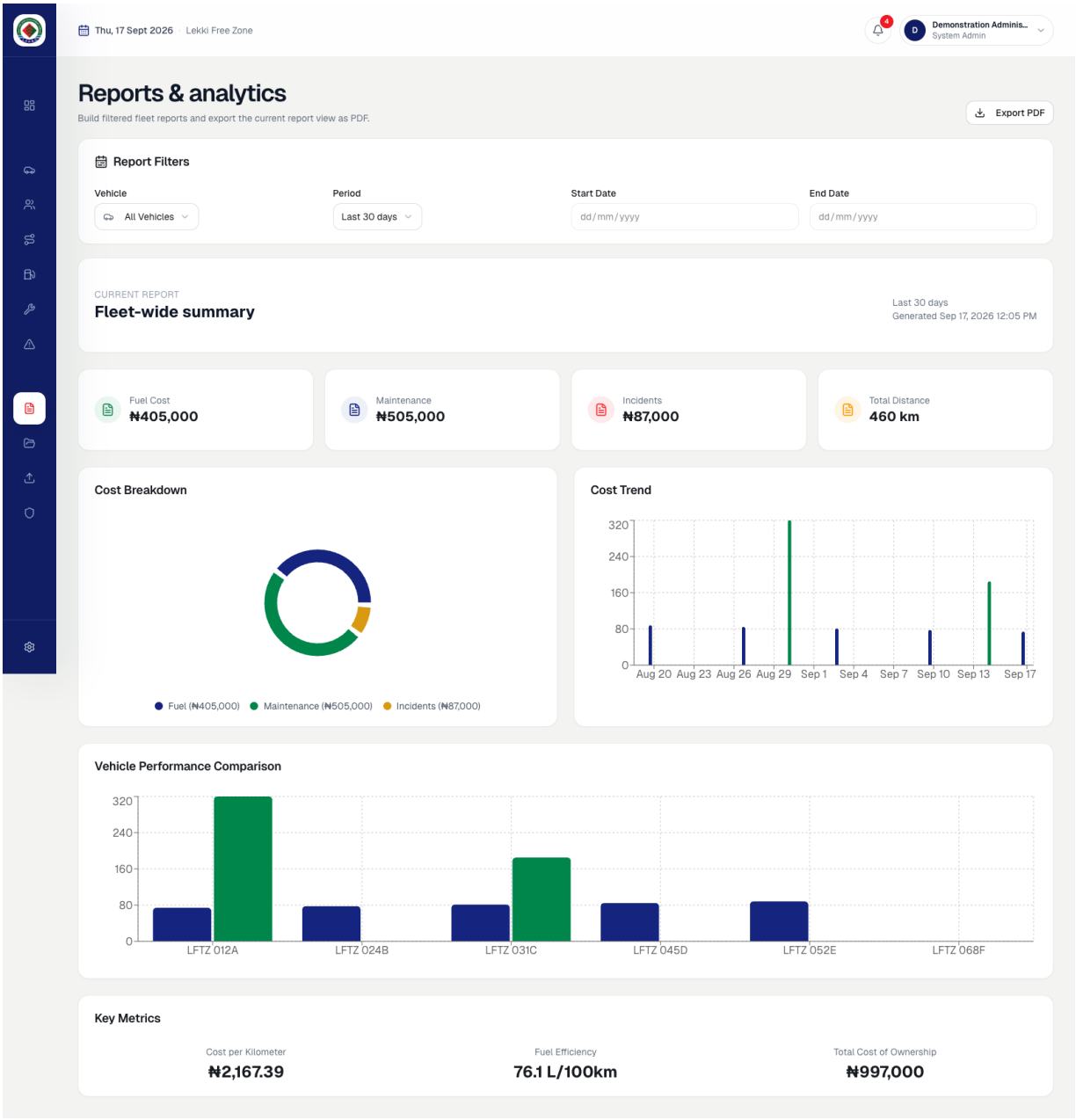


Figure 4.5: Reports and analytics interface using fictional demonstration records. Source: Author's implementation, 2026.

4.10 Document and Notification Modules

Document metadata identifies the related vehicle or driver, category, expiry information and Storage object path. Files are uploaded to the documents bucket and associated metadata is written only after upload succeeds. In-app notifications use deterministic identifiers for automatically generated alerts so repeated checks update an existing warning instead of producing uncontrolled copies. Optional email delivery records per-recipient outcomes to avoid repeated sends.

4.11 User Administration

The user access page is available only to system administrators. Account creation and deletion are delegated to deployed Edge Functions because these operations require protected Auth administration. A system administrator can assign canonical roles, while safeguards prevent self-deletion and removal of the last administrator. The service-role credential is available only in the hosted function environment.

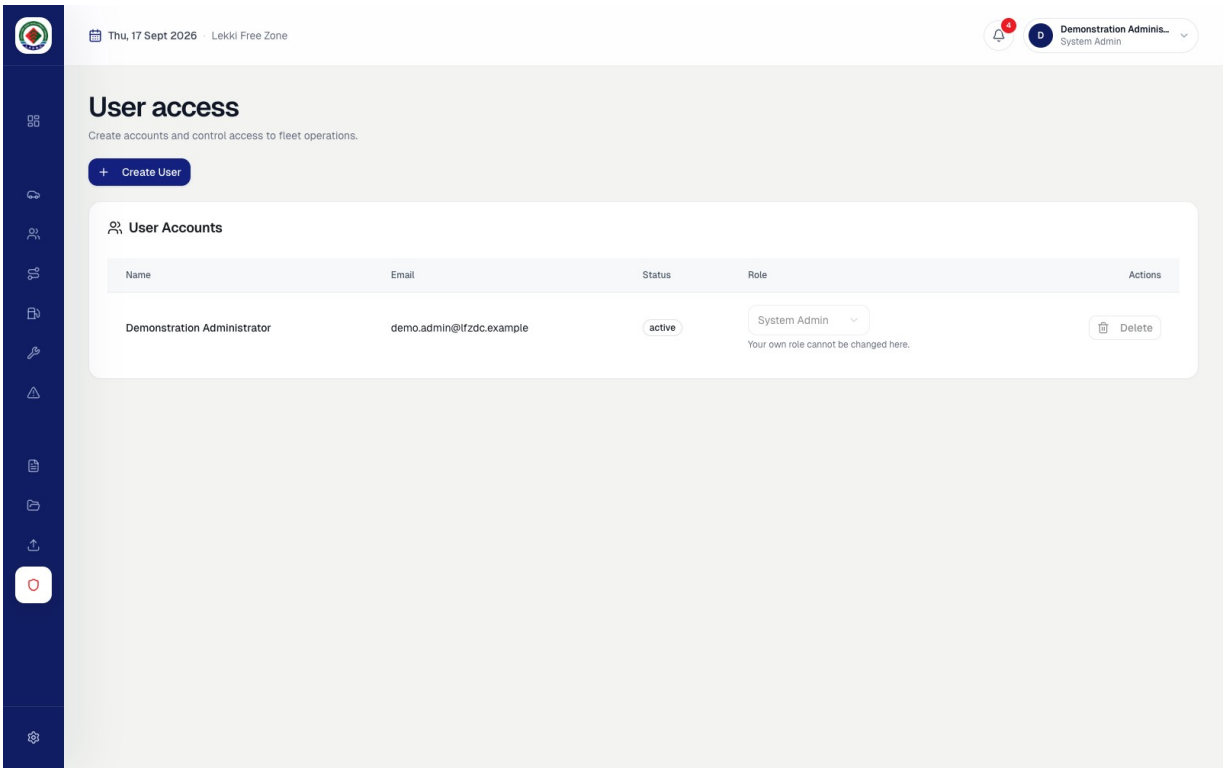


Figure 4.6: User access administration interface using a fictional account. Source: Author's implementation, 2026.

4.12 Deployment and Portable Submission

The frontend can be deployed as static Vite output on Vercel and communicates with the configured Supabase project. For academic submission, the source code includes a safe environment example, setup commands, backend documentation, API documentation and a numbered SQL database package. The hosted database itself is not copied to a flash drive; instead, the submitted SQL scripts recreate the schema, policies, storage configuration and fictional sample data in a PostgreSQL-compatible Supabase project.

4.13 Selected Interface Source Code

The following excerpts connect the visible interfaces to their implementation. They are limited to code that directly controls page composition, user feedback and task progression; complete source

files remain available in the submitted codebase. The editor-style presentation includes the original file path and line numbers so an assessor can locate each excerpt quickly.

Dashboard interface composition — src/pages/Dashboard.tsx

```
213   return (
214     <div className="space-y-5 pb-8">
215       <section className="lfz-enter flex flex-col gap-5 xl:flex-row xl:items-end xl:justify-...
216         <div>
217           <h1 className="text-3xl font-semibold tracking-[-0.035em] text-[#171a1f] sm:text-[...
218             Fleet overview
219           </h1>
220           <p className="mt-2 max-w-2xl text-sm leading-6 text-gray-500">
221             Vehicle availability, monthly spending and outstanding service work.
222           </p>
223         </div>
224         <div className="flex flex-wrap gap-2.5">
225           {canManageRecords ? (
226             <Button asChild variant="outline" className="h-10 border-gray-300 bg-white px-4 ...
227               <Link to="/import">Import records</Link>
228             </Button>
229           ) : null}
230           <Button asChild className="h-10 px-4">
231             <Link to="/vehicles">
232               View vehicles
233               <ArrowUpRight className="h-4 w-4" />
234             </Link>
235           </Button>
236         </div>
237       </section>
238
239       <section className="lfz-enter lfz-enter-delay-1 grid gap-3 sm:grid-cols-2 xl:grid-cols...
```

Figure 4.7: Dashboard heading, actions and summary-grid composition from src/pages/Dashboard.tsx. Source: Author's implementation, 2026.

The dashboard excerpt shows that summary cards are implemented as navigable controls. Permission-aware actions are rendered only for users who can manage records, while the layout changes across screen sizes through responsive grid classes.

Vehicle validation and save flow — src/pages/Vehicles.tsx

```
117   const handleSubmit = async (e: React.FormEvent) => {
118     e.preventDefault();
119     setIsSubmitting(true);
120
121     try {
122       // Validation
123       if (!formData.plateNumber || !formData.vehicleType || !formData.brand || !formData.mod...
124         toast.error('Please fill in all required fields');
125         setIsSubmitting(false);
126         return;
127       }
128
129       // VIN validation (17 characters)
130       if (formData.vin && formData.vin.length !== 17) {
131         toast.error('VIN must be exactly 17 characters');
132         setIsSubmitting(false);
133         return;
134       }
135
136       // Check for duplicate plate number
137       const existing = await findVehicleByPlateNumber(formData.plateNumber);
138       if (existing) {
139         toast.error('A vehicle with this plate number already exists');
140         setIsSubmitting(false);
141         return;
142       }
      // ...
163       await createVehicle(newVehicle);
164       toast.success('Vehicle added successfully');
165       setIsAddModalOpen(false);
166       resetForm();
167       await loadVehicles();
```

Figure 4.8: Required-field, VIN and duplicate-plate checks followed by record creation and interface refresh. Source: Author's implementation, 2026.

The vehicle handler performs client-side validation before calling the data layer. A successful write closes the form, clears its state, reloads the register and gives immediate feedback. Database constraints and policies remain the final protection beyond these interface checks.

Three-stage import interface — src/pages/DataImport.tsx

```
312       <ol className="grid grid-cols-3 gap-2 border-y border-gray-200 py-4 sm:flex sm:gap-8" ...
313         {['Choose file', 'Review', 'Import'].map((step, index) => {
314           const currentStep = result ? 2 : preview ? 1 : 0;
315           return (
316             <li key={step} aria-current={index === currentStep ? 'step' : undefined} classNa...
317               <span className={`flex h-6 w-6 items-center justify-center rounded-full text-x...
318               {step}
319             </li>
320           );
321         })}
322       </ol>
    // ...
362           <div className="flex flex-wrap gap-3">
363             <Button onClick={handlePreview} disabled={!selectedFile || isParsing || isImport...
364               <RefreshCcw className={`h-4 w-4 ${isParsing ? 'animate-spin' : ''}`} />
365               {isParsing ? 'Checking records...' : 'Review records'}
366             </Button>
367             <Button
368               variant="outline"
369               disabled={!preview?.canImport || isParsing || isImporting || Boolean(result)}
370               onClick={handleImport}
371               className="gap-2"
372             >
373               <Database className="h-4 w-4" />
374               {isImporting ? 'Importing...' : result ? 'Import complete' : 'Import records'}
375             </Button>
376           </div>
377
378           <p className="text-sm text-gray-500">Duplicate records are skipped. Nothing is sav...
```

Figure 4.9: Three-stage import progress indicator and guarded review/import actions from src/pages/DataImport.tsx. Source: Author's implementation, 2026.

The import excerpt demonstrates how the visible stage is derived from preview and result state. Buttons remain unavailable until the required file or valid preview exists, reinforcing the rule that records are reviewed before any database write.

CHAPTER FIVE

TESTING AND EVALUATION

This chapter records the verification approach, representative tests, results, usability assessment and limitations.

5.1 Testing Strategy

Testing combined source verification, functional module checks and security-oriented behavior. Linting and TypeScript compilation were used to detect invalid code and type errors. A production build was used to verify deployable assets. Functional tests covered authentication, authorization, creation and status workflows, report filters, import safety and responsive navigation. Backend checks were performed with RLS enabled and with fictional operational data.

5.2 Test Cases and Outcomes

ID	Test	Expected result	Outcome
T01	Valid staff sign-in	Session and profile load; protected dashboard opens.	Pass
T02	Invalid sign-in	Access is denied and a clear message is shown.	Pass
T03	MFA-protected account	Six-digit authenticator challenge is required.	Pass
T04	End user opens an editor action	Action is hidden/blocked and database policy prevents write.	Pass
T05	Create vehicle with duplicate plate	Duplicate is rejected before creation.	Pass
T06	Assign an already assigned driver	Conflicting active assignment is closed.	Pass
T07	Submit a lower odometer reading	History is recorded but current reading does not decrease.	Pass
T08	Start active maintenance	Vehicle readiness reflects under-maintenance state.	Pass
T09	Preview workbook with invalid rows	Errors identify sheet/row and import remains blocked.	Pass
T10	Import repeated workbook rows	Configured duplicates are skipped and counted.	Pass
T11	Filter reports by period/vehicle	Metrics and charts recalculate for the selected range.	Pass
T12	Call Edge Function without a session	Request returns an unauthorized response.	Pass
T13	System admin attempts self-delete	Operation is refused.	Pass
T14	Responsive navigation on narrow screen	Icon navigation remains usable and content fits.	Pass

5.3 Build and Source Verification

The submission workflow includes an npm check command that runs the linter and production build. The reviewed codebase contains no submitted secret values and retains only the UI components used by the current application. Backend and API documentation were updated to match the current role names, modules, SQL package and three deployed Edge Functions.

5.4 Security Evaluation

Security was evaluated across interface, database and server-function boundaries. The application does not assume that hidden UI controls are sufficient. RLS remains enabled on application tables, service-role credentials are excluded from the browser, and each privileged function verifies both the access token and the application profile. Unauthenticated requests to the deployed functions were confirmed to return HTTP 401 responses. Password-reset behavior accounts for Supabase's AAL2 requirement when MFA is enabled.

5.5 Usability Evaluation

The interface was refined to reduce the "generic dashboard" appearance and to improve operational clarity. Changes included LFZDC-aligned color usage, a compact expandable navigation rail, consistent icon treatment, reduced decorative tags, direct page titles, more restrained cards and a shorter import process. The final import screen explains that Google Sheets should first be exported to Excel, shows that nothing is saved during review, and separates the process into only three stages.

5.6 Limitations

- Hosted operation requires internet access to the configured Supabase project.
- The current public document bucket is suitable for the demonstrated workflow but sensitive production documents should use private signed URLs.
- Automated unit and end-to-end coverage can be expanded beyond the current functional and build checks.
- Email notifications require correct hosted SMTP secrets and are intentionally best-effort.
- The system does not integrate with live GPS, telematics or external licensing services.

CHAPTER SIX

USER AND INSTALLATION GUIDE

This chapter explains how a lecturer or authorized operator can run, access and use the submitted system.

6.1 Requirements

- A computer running a current version of Windows, macOS or Linux.
- Node.js version 20 or newer and npm.
- A modern browser such as Chrome, Edge, Firefox or Safari.
- Internet access for the configured hosted Supabase backend.
- A test user account with a matching profile record.

6.2 Running the Submitted Code

1. Copy the project folder from the flash drive to a writable folder on the computer.
2. Open a terminal in the project folder.
3. Run “npm install” to install the exact dependencies recorded in package-lock.json.
4. Create .env.local from .env.example and enter the supplied Supabase project URL and publishable anonymous key. Do not place a service-role key in this file.
5. Run “npm run dev”.
6. Open the local address shown in the terminal, normally http://localhost:5173.

For a production-style local check, run “npm run build” and then “npm run preview”. The source-quality check is “npm run check”. If a lecturer is testing only the already deployed version, local installation is not required; valid test credentials and internet access are sufficient.

6.3 Signing In

1. Open the application login page.
2. Enter the assigned staff email address and password.
3. Use the eye icon if it is necessary to confirm the typed password in a private environment.
4. Select Sign In.
5. If the account has MFA enabled, enter the six-digit code from the enrolled authenticator application.

Use “Forgot password?” to request a recovery email. A user with MFA must verify the authenticator code before changing the password. Recovery links expire and should not be reused.

6.4 Navigation

The left navigation rail groups the system into Overview, Fleet Operations, and Records & Control. When collapsed, the icons remain visible. Moving to the rail expands it for labels, and the pin control can keep it open where available. The top bar shows the current date, Lekki Free Zone context, notifications and the signed-in account menu.

6.5 Dashboard

1. Open Overview to see current fleet availability and expenditure.
2. Select a summary card to open the related filtered register.
3. Review Fleet Readiness before assigning vehicles to new work.
4. Use Recent Notices to investigate service, document or licence warnings.

6.6 Managing Vehicles and Drivers

Authorized administrators and editors can add records. Required vehicle fields include plate number, type, brand and model. VIN values, when provided, must contain the expected 17 characters. Driver records require name, phone, licence number and licence expiry. Use status changes instead of deletion so that trips, assignments and incidents retain their history. Select the eye action to open a detail page and review related activity.

1. Open Vehicles and select Add Vehicle. The action is available only to an editor or system administrator.
2. The form opens over the vehicle register. Fields marked with an asterisk are required; the current odometer and status can also be recorded at this stage.

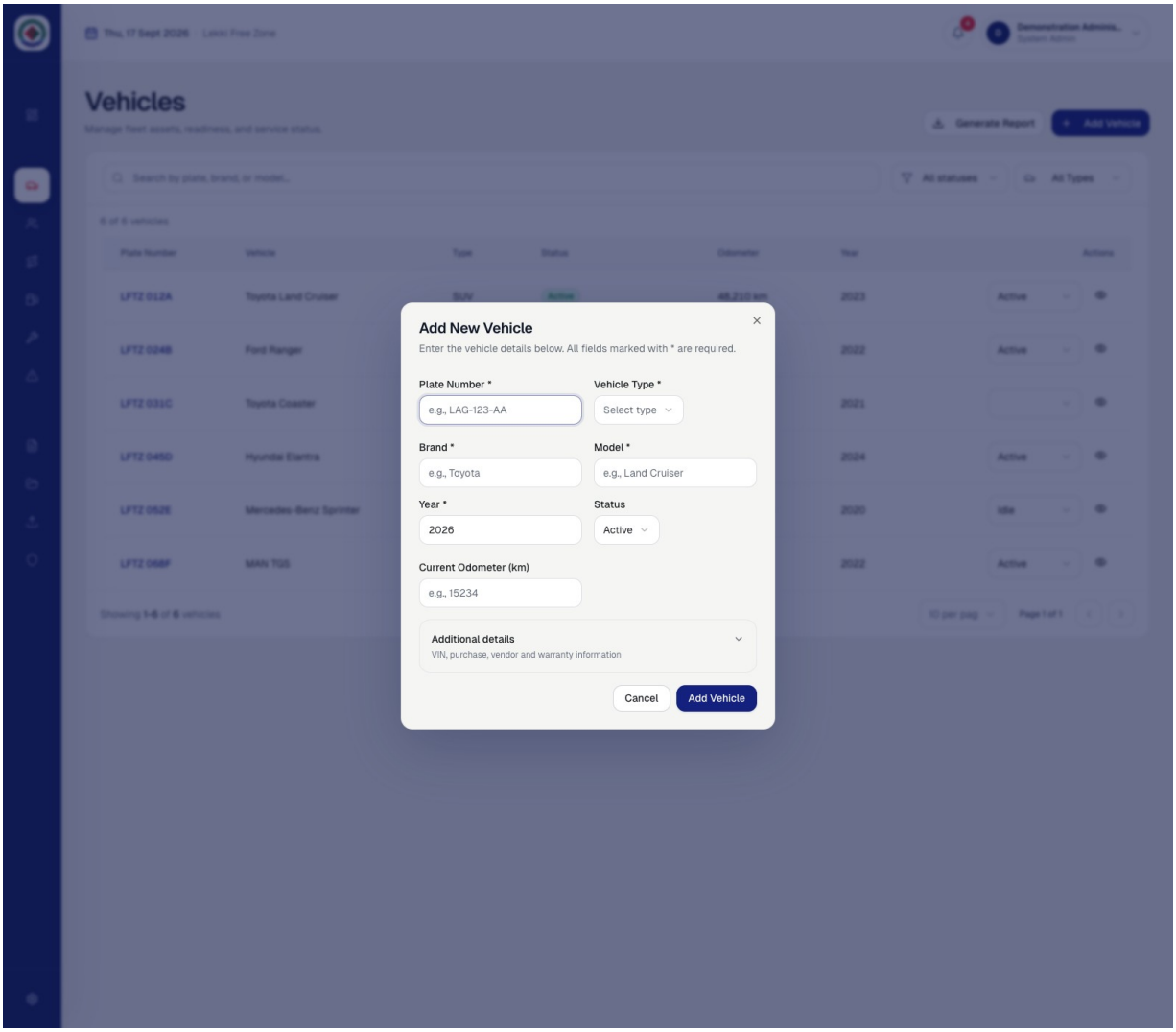


Figure 6.1: Add New Vehicle form before data entry. Source: Author's implementation using fictional demonstration data, 2026.

Enter the plate number, vehicle type, brand, model and year. Expand Additional details when the VIN, engine number, purchase date, purchase cost, vendor or warranty expiry is available. The completed example below shows both the required and optional fields before submission.

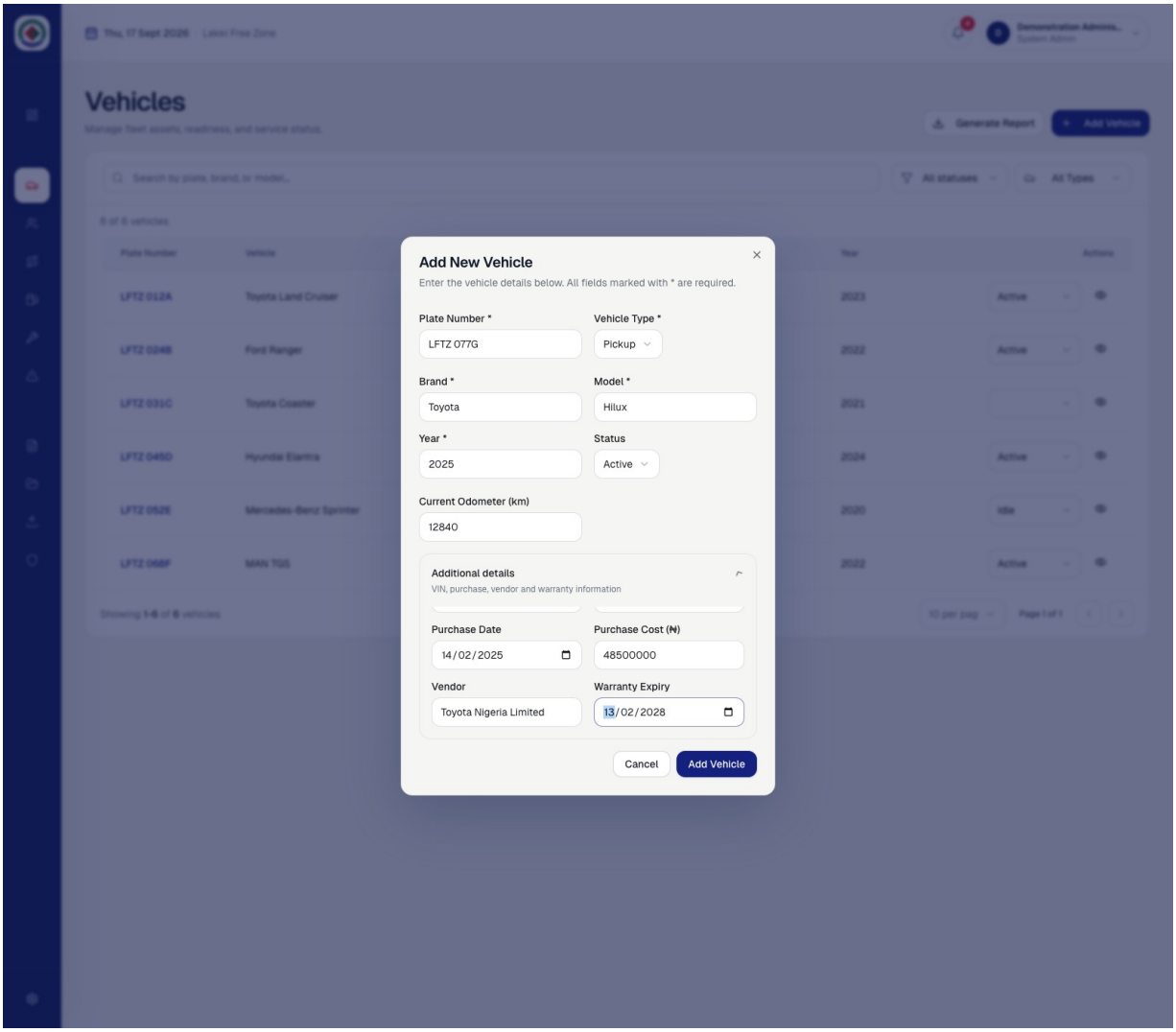


Figure 6.2: Completed vehicle form with the Additional details section expanded. Source: Author's implementation using fictional demonstration data, 2026.

1. Review the values and select Add Vehicle.
2. The interface checks required values, the optional VIN length and whether the plate number already exists.

After a successful save, the dialog closes and the register refreshes. Confirm that the vehicle count has increased and that the new plate, vehicle description, type, status, odometer and year appear correctly. The fictional example LFTZ 077G is shown on the final row below.

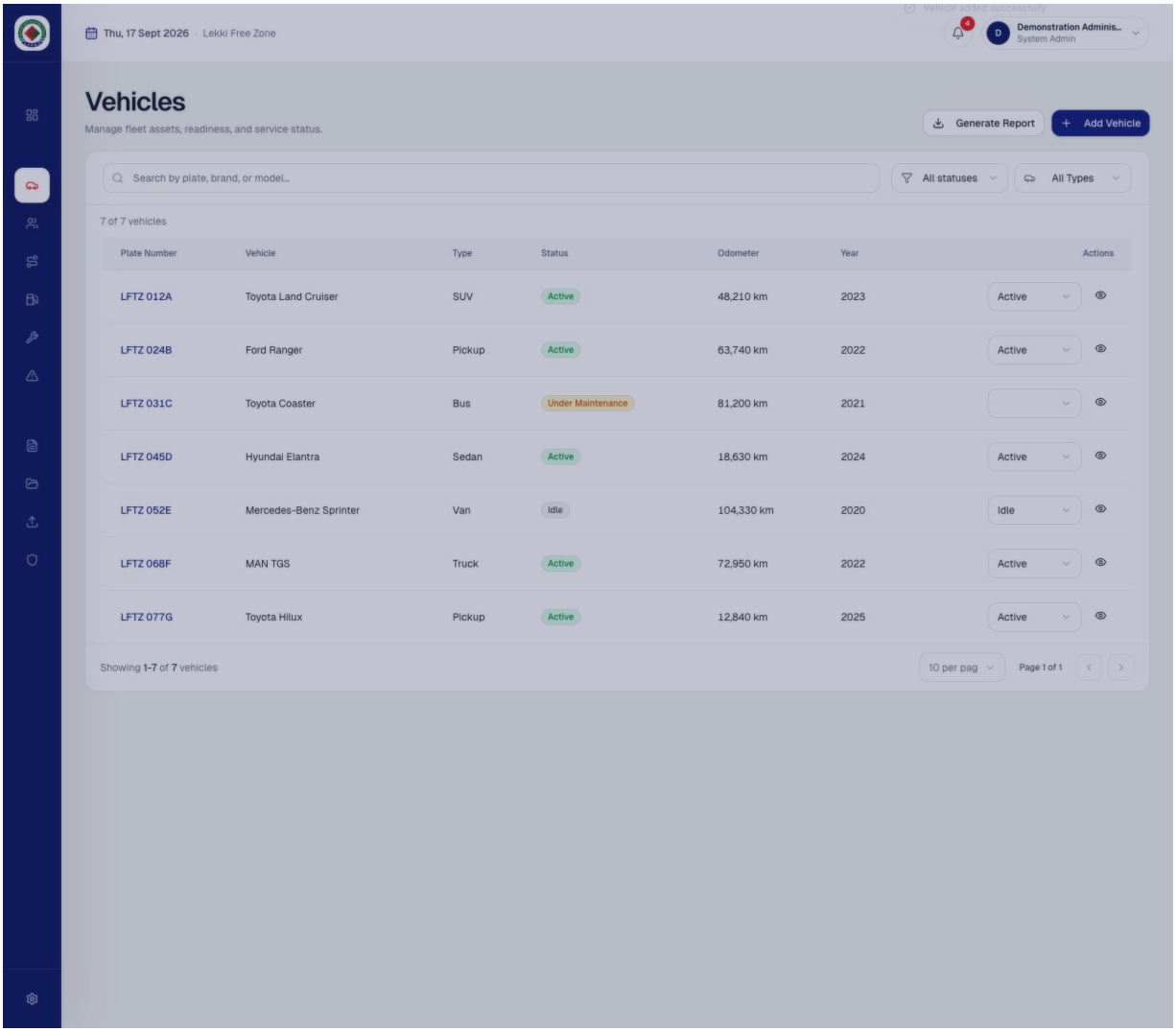


Figure 6.3: Newly added vehicle visible in the refreshed register. Source: Author's implementation using fictional demonstration data, 2026.

Use the eye action on the new row to open its details and verify linked activity. If the record is incorrect, update it through the supported edit or status controls rather than creating a duplicate plate.

To register a driver, open Drivers and select Add Driver. Enter the required name, phone number, licence number and licence expiry. The Additional details section records optional contact, employment and address information. Review the entries before selecting Add Driver.

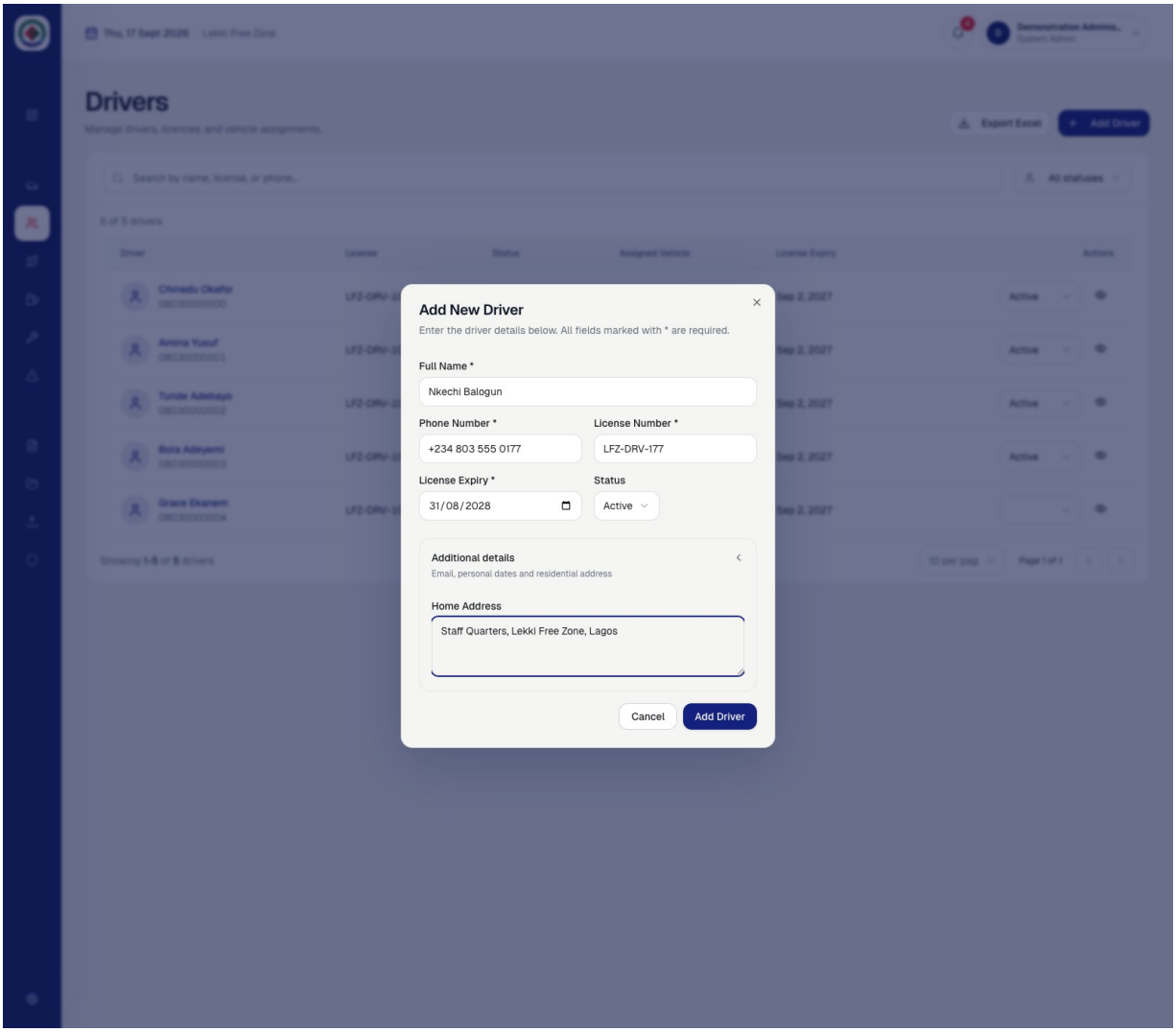


Figure 6.4: Completed driver registration form with Additional details expanded. Source: Author's implementation using fictional demonstration data, 2026.

After submission, confirm that the driver count increases and the new record appears with the correct licence, status and expiry date. A vehicle may be assigned later through the controlled assignment workflow; the register shows a dash until an active assignment exists.

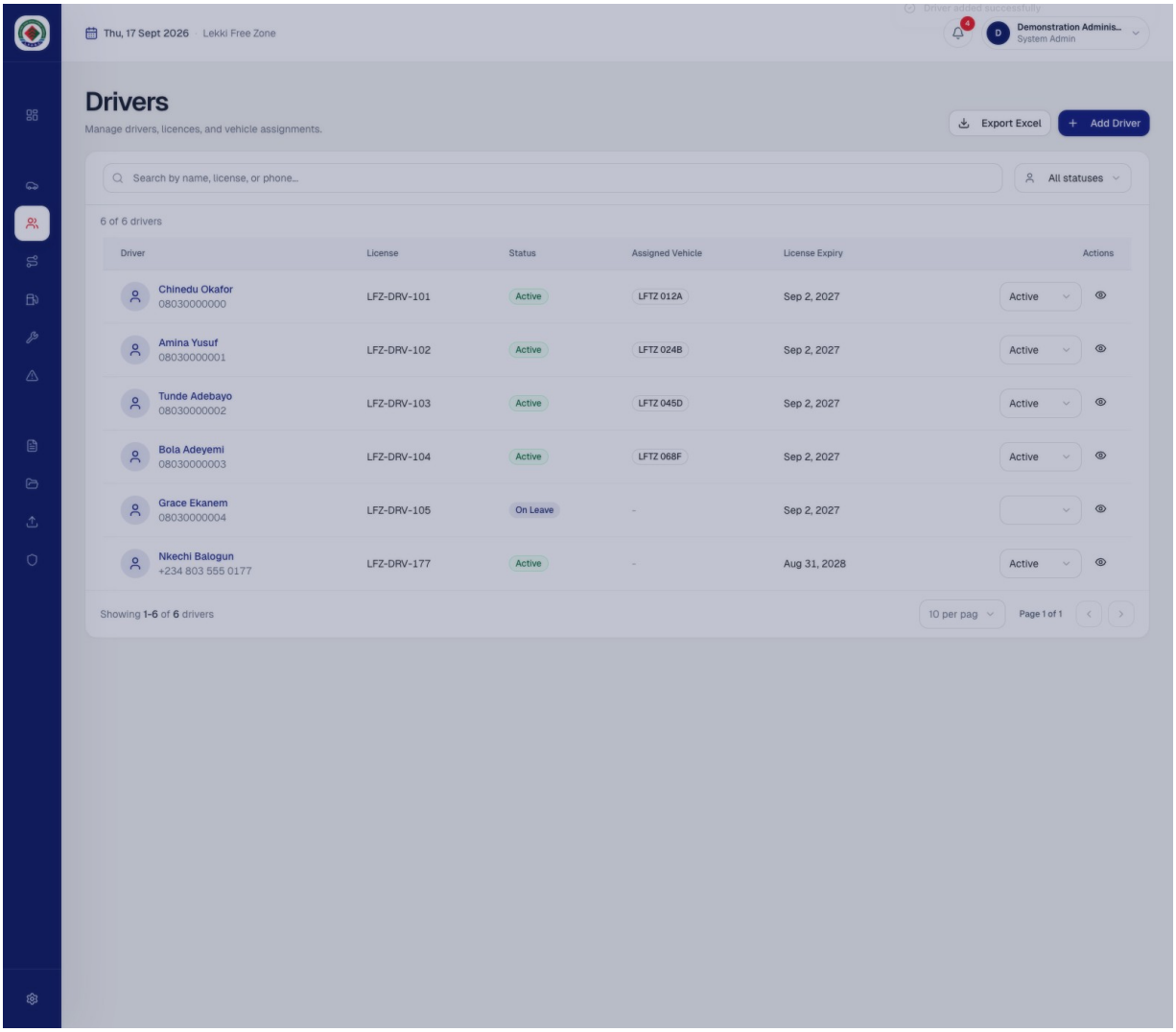


Figure 6.5: Newly added driver visible in the refreshed driver register. Source: Author's implementation using fictional demonstration data, 2026.

6.7 Recording Operational Activity

- Trips: select a vehicle and driver, enter date, route, distance and purpose.
- Fuel: enter quantity, cost, station and odometer; the vehicle mileage advances only when appropriate.
- Maintenance: select service type, schedule, provider, cost and status; complete the record when work finishes.
- Incidents: record title, date, severity, location, description and repair cost; update resolution details later.
- Documents: select the related vehicle or driver, category and file; use accurate expiry information when applicable.

The following worked examples demonstrate how an operator records a trip, schedules maintenance and reports an incident. Each example uses fictional values and shows both the completed form and the resulting register.

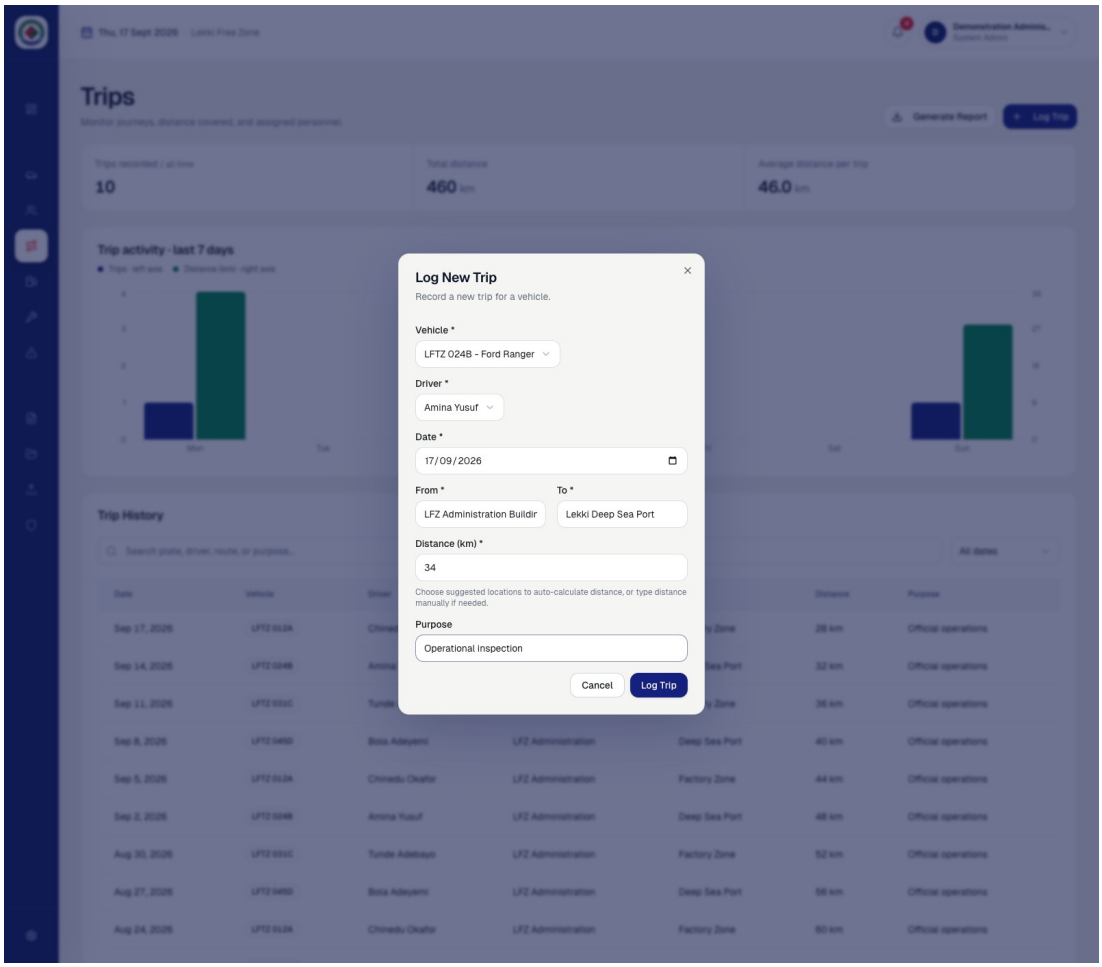


Figure 6.6: Completed trip form showing the selected vehicle, driver, route, distance and purpose. Source: Author's implementation using fictional demonstration data, 2026.

Select Log Trip after checking the route information. The system closes the form, updates the trip totals and places the saved journey in Trip History. Confirm the date, vehicle, driver, origin, destination, distance and purpose.

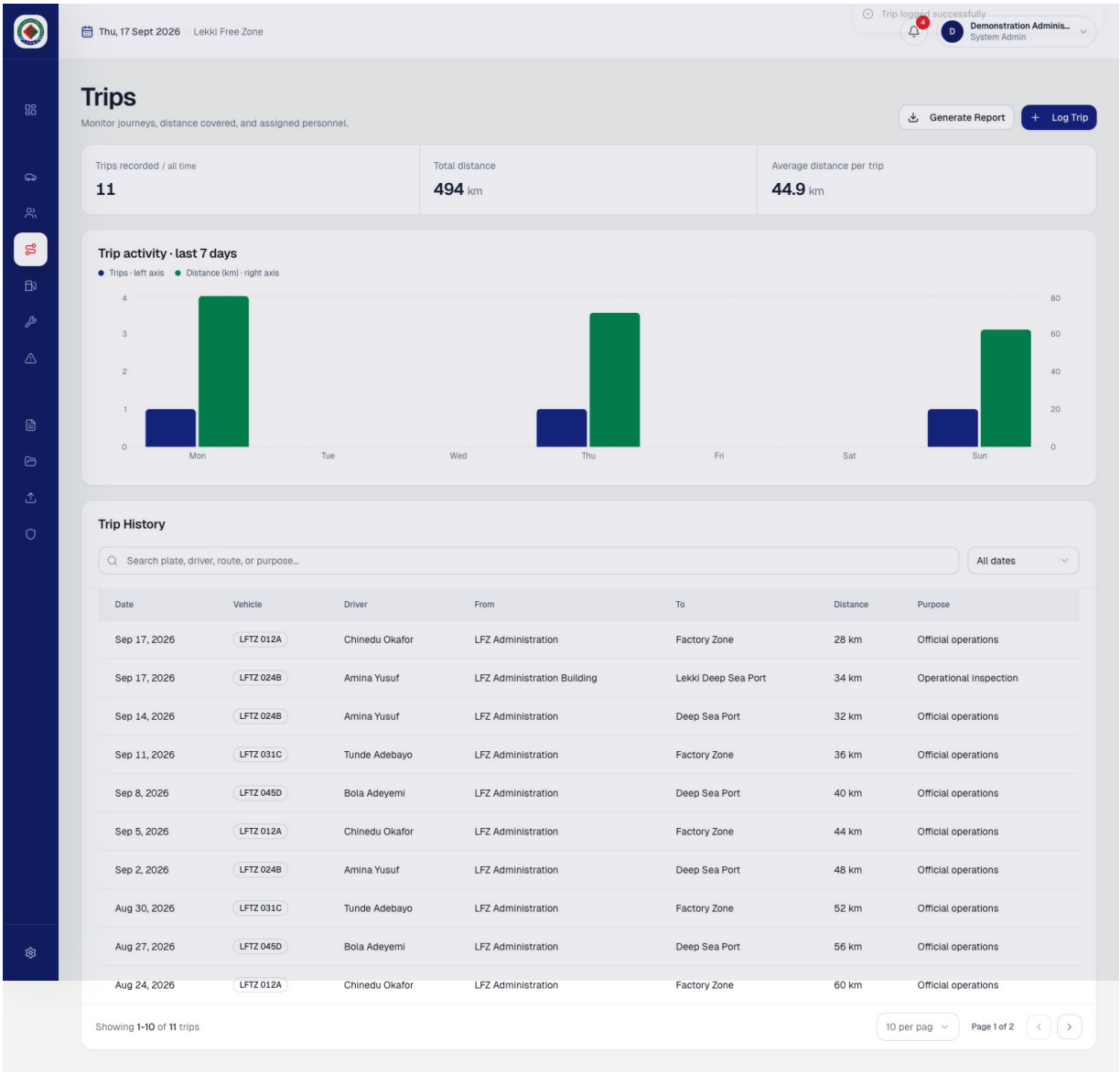


Figure 6.7: Logged trip visible in Trip History and included in the updated summary. Source: Author's implementation using fictional demonstration data, 2026.

To plan workshop activity, open Maintenance and select Schedule Maintenance. Choose the vehicle and service type, enter the scheduled date and estimated cost, and add the provider and work description where known.

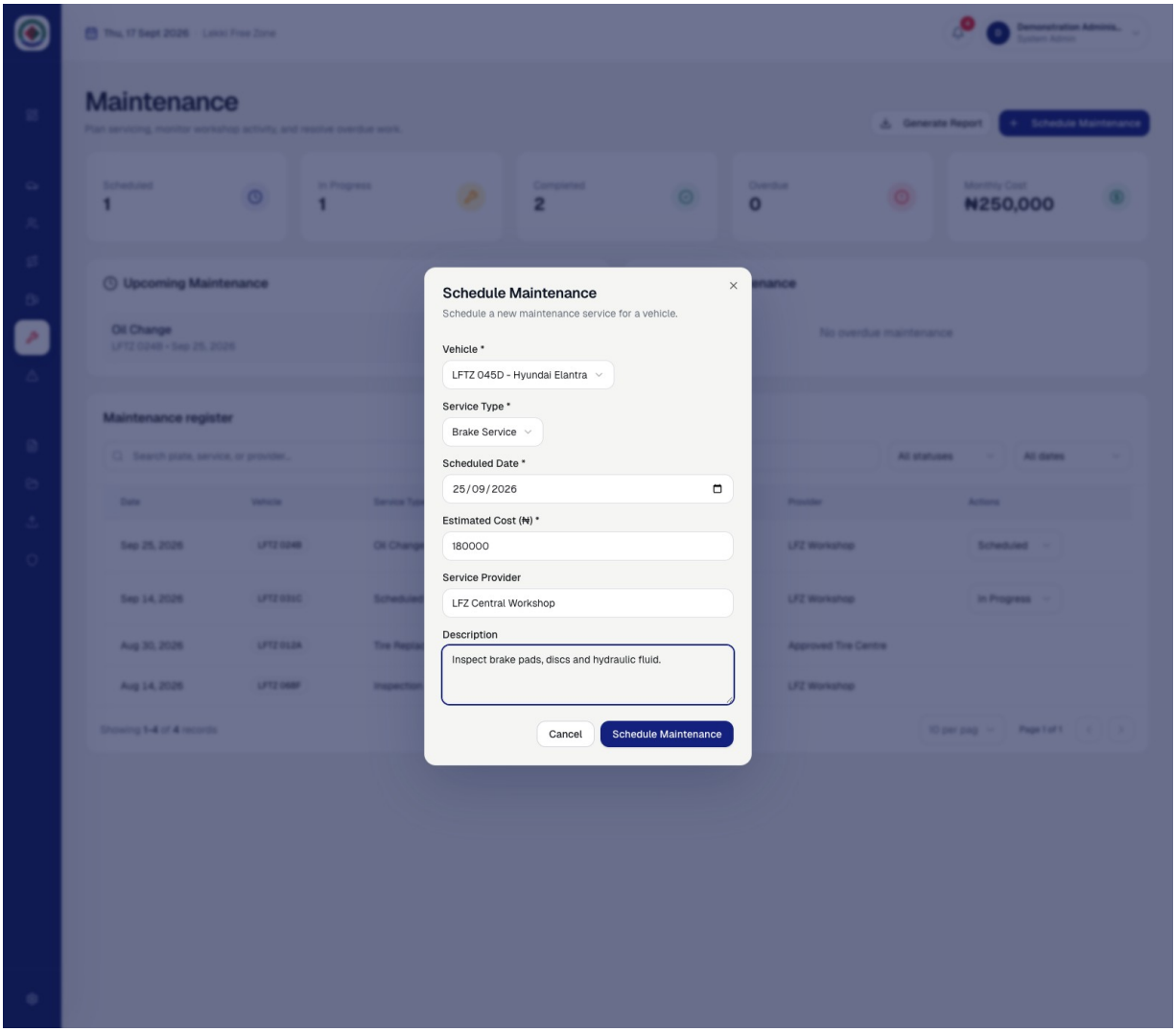


Figure 6.8: Completed maintenance scheduling form. Source: Author's implementation using fictional demonstration data, 2026.

After saving, verify that the scheduled count and monthly cost reflect the new work and that the maintenance register contains the correct vehicle, service, date, cost and provider. The status can later move from Scheduled to In Progress and Completed.

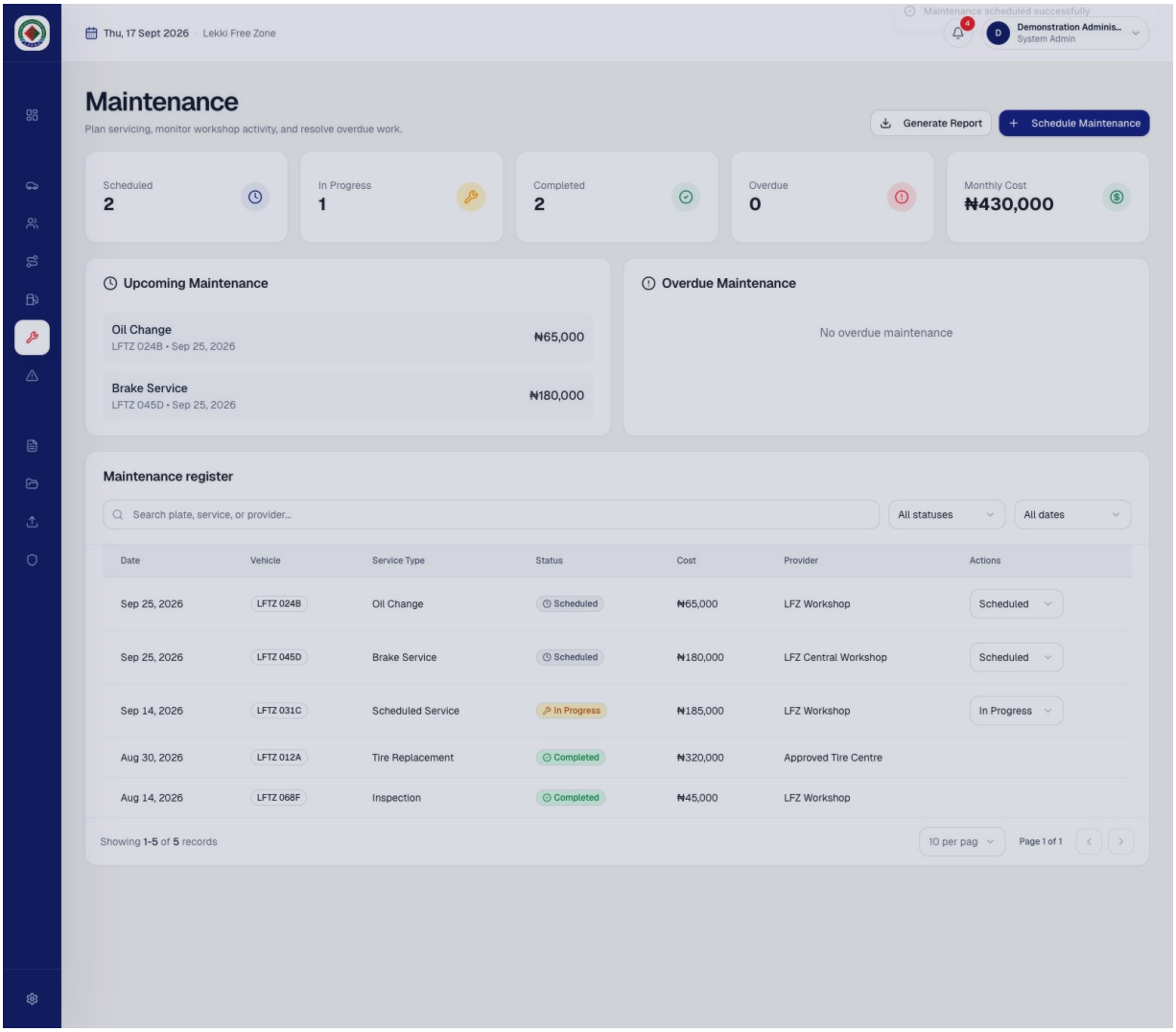


Figure 6.9: Scheduled work visible in the maintenance summary and register. Source: Author's implementation using fictional demonstration data, 2026.

To record an accident or operational issue, open Incidents and select Report Incident. Choose the vehicle and, where applicable, the driver. Enter a clear title, date, severity, location, estimated repair cost and factual description before submission.

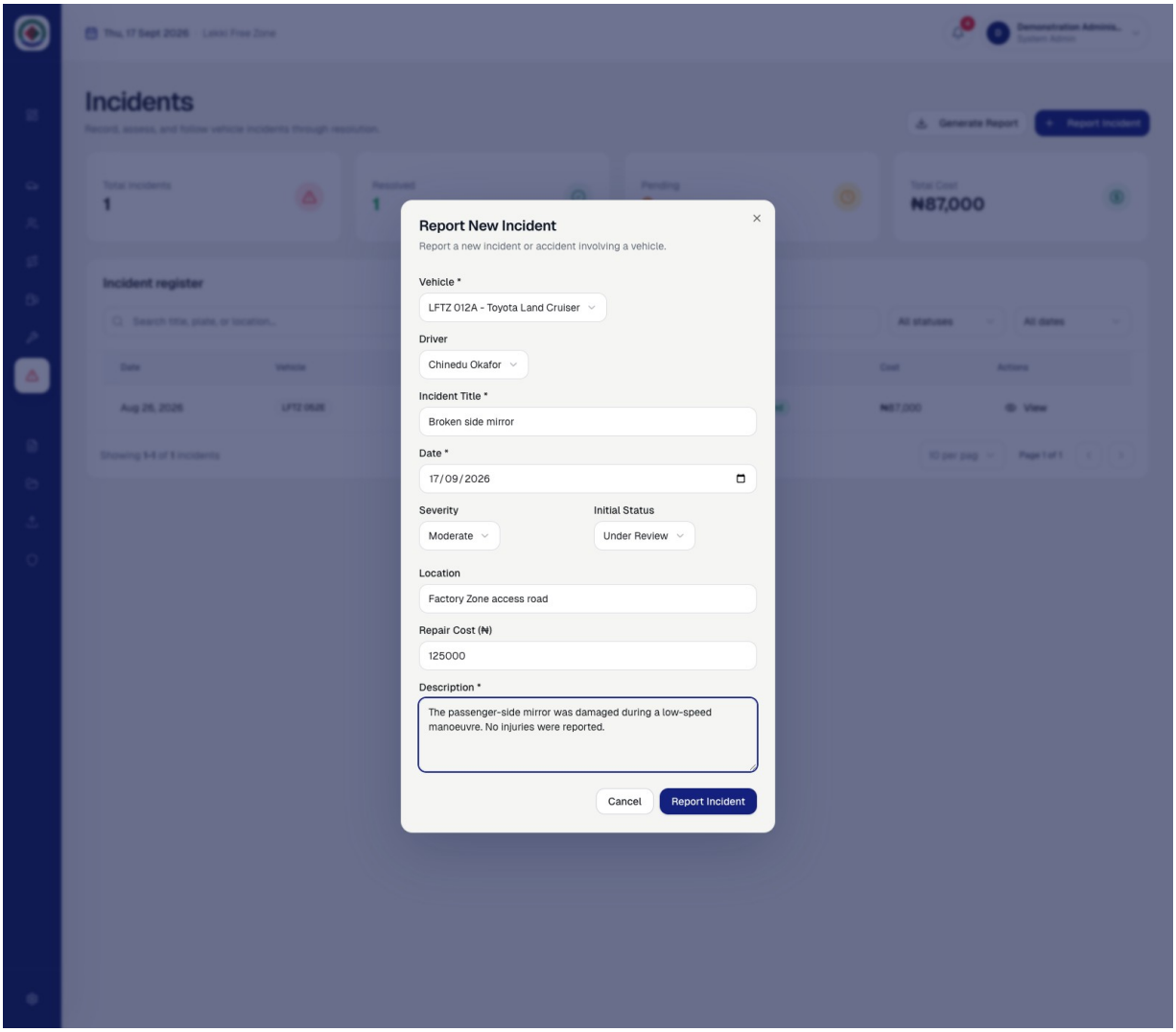


Figure 6.10: Completed incident report form before submission. Source: Author's implementation using fictional demonstration data, 2026.

After submission, confirm that the incident totals and cost summary update and that the new record appears in the register. Open View to review the full report, update its status through repair, and record the final repair cost when known.

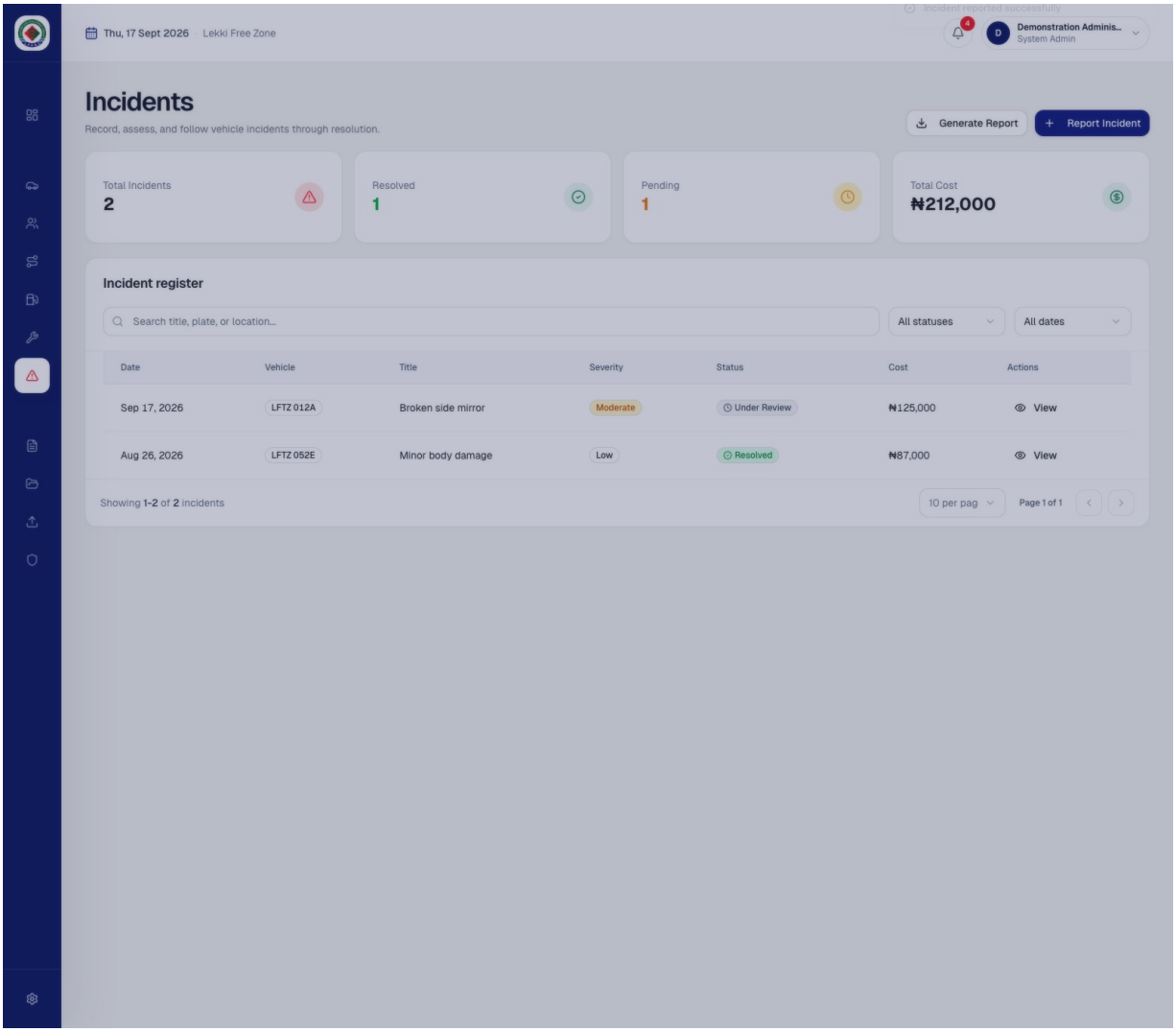


Figure 6.11: Reported incident visible in the incident summary and register. Source: Author's implementation using fictional demonstration data, 2026.

6.8 Importing Existing Records

1. Keep an untouched backup of the original workbook.
2. If the records are in Google Sheets, download the workbook as Microsoft Excel (.xlsx).
3. Open Data Import and optionally download the standard template.
4. Choose the Excel file and select Review records.
5. Check detected sheets, field mappings, counts, vehicle/driver links and reported row issues.
6. Correct blocking errors in the workbook or mapping and review again.
7. Select Import records only when the preview is acceptable.
8. Review the completion summary and import history. Repeating the same file may produce duplicate skips.

A small pilot workbook should be imported before a large archive. Empty supported sheets are acceptable. The importer creates missing vehicles and drivers where supported, links operational rows by plate and licence identifiers, and does not save the original workbook file to the database.

6.9 Reports and Exports

Open Reports, select a vehicle or All Vehicles, and choose a preset period or custom dates. The summary cards and charts recalculate for that filter. Use Export PDF to save the current report view. Vehicle and other registers may also provide Excel reports for the current filtered view or a named operational condition.

6.10 Notifications and Settings

The notification icon displays unread operational alerts. Open an alert to investigate the associated module and mark items as read after review. Settings control interface and notification preferences. Profile controls allow the signed-in user to update supported personal details, password, email and TOTP MFA configuration. Sensitive changes may require recent MFA verification.

6.11 User Administration

Only a system administrator can open User Access. Create User provisions both the Supabase Auth account and profile. Assign the least-privileged appropriate role. Deleting a user removes the Auth account and profile only when the function safeguards permit it. The current administrator cannot delete their own account, and the final system administrator cannot be removed.

6.12 Troubleshooting

Problem	Recommended response
Login fails	Confirm the email/password, internet connection, active profile status and Supabase Auth health.
MFA code rejected	Use the current six-digit authenticator code and confirm the device time is automatic.
Password reset says AAL2 required	Complete the authenticator challenge before saving the new password.
Application loads but data does not	Verify .env.local values, internet access, the profile row and RLS policies.
Workbook cannot be imported	Use .xlsx/.xls, review sheet mappings and correct blocking row errors.
Duplicate rows are skipped	Confirm that the same workbook or identifiers were not previously imported.
Document upload fails	Check file selection, authenticated access and Storage bucket policies.
User creation/deletion fails	Confirm system_admin access and that the deployed Edge Functions are available.

6.13 Safe Submission and Demonstration

- Do not include .env.local, service-role keys, database passwords, SMTP passwords or personal credentials on the flash drive.
- Provide test credentials separately and change or disable them after assessment.
- Use the database-submission folder for schema inspection and restoration.
- Use only fictional sample data during a public defence or classroom demonstration.
- Keep the original project folder backed up before making last-minute presentation changes.

CHAPTER SEVEN

CONCLUSION AND RECOMMENDATIONS

This chapter summarizes the project outcome and identifies practical extensions.

7.1 Conclusion

The project achieved its aim of creating a centralized web-based Fleet Management System for Lekki Free Zone Development Company. The completed application brings vehicle, driver, assignment, trip, fuel, maintenance, incident, document, notification, report and historical import workflows into one coherent interface. Its design connects operational records instead of treating them as isolated forms, while role controls and database policies protect modification rights.

A major outcome is the treatment of historical migration as an operator-facing process. Existing Excel and exported Google Sheets records can be reviewed, mapped and validated before any write occurs. The final interface also demonstrates that a functional academic system can use a restrained brand identity and task-focused layouts rather than a generic collection of dashboard cards.

The source code, backend documentation, API reference and numbered database package make the system suitable for assessment and future maintenance. Hosted Supabase remains appropriate because it provides PostgreSQL, authentication, storage and server functions, while the SQL package ensures that the database design is still portable and inspectable.

7.2 Recommendations

- Convert the document bucket to private access with short-lived signed URLs before storing sensitive production files.
- Add automated unit, integration and end-to-end tests to the existing build verification.
- Introduce audit-log tables for high-risk administrator and record-status changes.
- Add scheduled server-side notification generation so alerts do not depend on a user opening the application.
- Integrate GPS or telematics only after defining data-retention, privacy and connectivity requirements.
- Add configurable approval workflows for high-cost maintenance or incident repairs.
- Periodically review indexes, connection use, RLS performance and backup restoration procedures.

7.3 Contribution of the Project

The project contributes a practical reference implementation for fleet record centralization in an organizational setting. It demonstrates how a modern frontend can communicate directly with a PostgreSQL-backed service without giving up authorization control, how privileged account operations can be isolated in server functions, and how legacy spreadsheet migration can be presented as a safe, comprehensible workflow for non-technical users.

REFERENCES

[1] React. (2026). Quick Start. https://react.dev/learn

[2] Vite. (2026). Getting Started. https://vite.dev/guide/

[3] PostgreSQL Global Development Group. (2026). Constraints. https://www.postgresql.org/docs/17/ddl-constraints.html

[4] PostgreSQL Global Development Group. (2026). Row Security Policies. https://www.postgresql.org/docs/17/ddl-rowsecurity.html

[5] Supabase. (2026). Database Overview. https://supabase.com/docs/guides/database/overview

[6] Supabase. (2026). Row Level Security. https://supabase.com/docs/guides/database/postgres/row-level-security

[7] Supabase. (2026). Securing Your Data. https://supabase.com/docs/guides/database/secure-data

[8] Supabase. (2026). Auth Architecture. https://supabase.com/docs/guides/auth/architecture

[9] Supabase. (2026). Securing Edge Functions. https://supabase.com/docs/guides/functions/auth

[10] OWASP Foundation. (2026). Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[11] SheetJS. (2026). SheetJS Community Edition Documentation. https://docs.sheetjs.com/

[12] LFZDC Fleet Management System source code, backend documentation and API reference (project submission, 2026).

APPENDIX A: DATABASE RESTORATION PACKAGE

The database-submission folder contains a numbered PostgreSQL/Supabase package. Apply the scripts in order to a new project. Authentication passwords and hosted files are excluded by design. After creating test Auth users, insert or update matching profiles with the same user UUIDs.

Order	File	Purpose
1	01-schema.sql	Core tables, functions, triggers and initial policies.
2	02-vehicle-odometer.sql	Current mileage fields and odometer history.
3	03-vehicle-service-mileage.sql	Next-service mileage tracking.
4	04-document-types.sql	Configurable document categories.
5	05-document-file-path.sql	Storage object path metadata.
6	06-profile-settings.sql	JSON user settings.
7	07-notification-email-deliveries.sql	Email delivery deduplication.
8	08-role-access-policies.sql	Canonical roles and RLS write policies.
9	09-storage-documents.sql	Document bucket and object policies.
10	10-sample-data.sql	Fictional academic demonstration data.

APPENDIX B: CUSTOM API ENDPOINTS

The frontend primarily uses Supabase Auth, PostgREST and Storage. Three custom POST endpoints provide server-side operations that should not be performed with browser credentials alone.

Endpoint	Authorized caller	Purpose
/functions/v1/create-user	system_admin	Create a confirmed Auth user and matching profile.
/functions/v1/delete-user	system_admin	Delete a target Auth user subject to administrator safeguards.
/functions/v1/send-notification-emails	Authenticated profile	Send an existing in-app alert to eligible email recipients.

All endpoints accept CORS preflight requests. Calls include the current access token. Each function verifies the token, confirms the profile and role where required, and returns a JSON error on failure. The service-role key and email credentials are read only from the hosted function environment.

APPENDIX C: SUBMISSION CHECKLIST

- Source folder opens and package.json is present.
- node_modules and local build output are not required on the flash drive.
- .env.example contains placeholders; .env.local is excluded.
- README.md explains installation, commands, structure and security notes.
- database-submission contains the numbered SQL package and restoration guide.
- docs contains current backend and API documentation.
- Supabase Edge Functions are present under supabase/functions and deployed to the selected project.
- This report is included in editable DOCX and reference PDF formats.
- Test credentials are supplied separately and contain no personal password.
- A backup copy of the final submission is retained outside the flash drive.